Hamiltonian s–t paths in solid grid graphs are decided in polynomial time: an algorithm with a Lean 4 proof

The author

2026-10-17

Released under CC0 1.0 (public domain dedication).

Abstract

A grid graph is a finite set of lattice points with every unit edge between two of its points. It is solid when it has no holes. We give an algorithm that decides, for a solid grid graph V and two distinct points $s, t \in V$, whether V has a Hamiltonian path from s to t. Correctness and a polynomial step bound are proved in Lean 4 with Mathlib. The main theorem `decideHamPath_main` states that the algorithm returns `true` exactly when a Hamiltonian s–t path exists, and that it takes at most $5\,220\,000\,(|V|+3)^6$ steps of explicit cost functions. Its only axioms are `propext`, `Classical.choice` and `Quot.sound`. The algorithm extends Sheffield's height-function algorithm for Hamiltonian cycles. Each endpoint is moved off the vertex set as a phantom. An endpoint with all eight king neighbours present becomes a hole, and its degree-one defect is a vortex of the height field. A slit and a sector label give a potential that rises along every climb, which replaces the height function in Sheffield's counting argument. The count behind a stuck phase is proved with columns, straight runs of rows through several cuts.

AI disclosure

Nearly the entirety of the work for this project was done by Claude (Opus 5.5), including the algorithm, writing this paper, writing the reference implementations in C, Python and JavaScript, and writing the Lean 4 proof. I, the listed author, contributed little more than the problem statement and guidance. I have listed myself as an author only to satisfy any requirement that a human author be attached to this work.

Contents

1 Introduction **2**

2 Overview **4**

3 Preliminaries **7**

3.1 Grid graphs, cells and solidity 7

3.2 Factors and pieces 7

3.3 Arcs and heights 8

3.4 Flips and cut cells 10

4 Sheffield's phase and the certificate **10**
4.1 Frames, critical cells and rows . . . 11
4.2 The phase . . . 12
4.3 Success branches and the undoing of bridges . . . 13
4.4 The certificate . . . 14
4.5 Columns and the stuck case . . . 14
4.6 Climbs . . . 15
4.7 The loop . . . 15

5 Phantom endpoints and the two-phantom core **16**

6 Deletion walks **17**

7 Inner endpoints: holes, vortices and slit heights **17**

8 Classes and the class search **19**

9 One inner endpoint **23**

10 Two inner endpoints **23**

11 The algorithm and its running time **25**

12 Formal verification **33**

13 Worked example **33**
13.1 One complete run . . . 33
13.2 Inside the leaves of a 4×4 instance . . . 35

14 The bulk case: endpoints far apart **36**

A Prior work and novelty **38**

B Lean module map **39**

C Proofs deferred from the main text **41**
C.1 Deletion (Lemma 6.1) . . . 41
C.2 The slit height (Theorem 7.4) . . . 41
C.3 Class implies `SolidFor` (Lemma 8.2) . . . 41
C.4 Strand pairing (Lemma 4.12) . . . 41
C.5 The step relation of a bridge . . . 41

D Sheffield's stuck case and columns **42**

E The starting factor as a maximum flow **43**

1 Introduction

Problem. A *grid graph* is given by a finite set $V \subset \mathbb{Z}^2$ of lattice points. Its edges are the unit segments between two points of V. A *Hamiltonian s–t path* is a path through every point

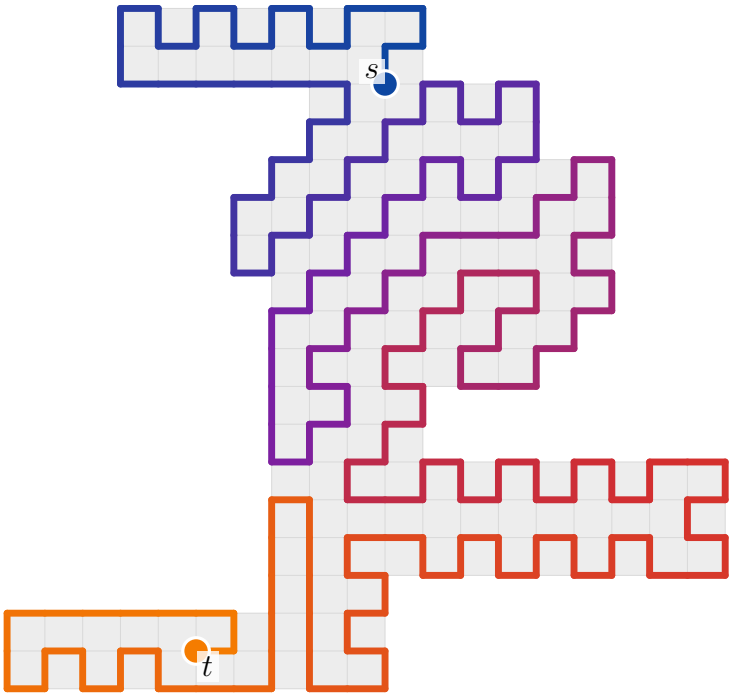


Figure 1: A Hamiltonian path from s to t on a solid grid graph with 197 points, found by the fast reference implementation of the algorithm of this paper, with its heuristic search turned off. The path is coloured along its length, from blue at s through purple and red to orange at t, so the order in which it visits the regions of the shape can be read off the colour. Both endpoints are inner, so the run goes through the two-hole case (Section 10). The shape was made by gluing rotated copies of rectangles, L, T, U and plus shapes, staircases and discs with arms at least 3 points wide.

of V exactly once, from s to t. For general grid graphs, deciding whether such a path exists is NP-complete [7]. A grid graph is *solid* when it has no holes (Definition 3.2). Umans and Lenhart gave a polynomial algorithm for Hamiltonian *cycles* in solid grid graphs [20]. Sheffield gave a second polynomial algorithm for the cycle problem based on height functions [17].

Result.

Theorem 1.1 (`decideHamPath_main`, `HPFinal.lean`)**.** *Let $V \subset \mathbb{Z}^2$ be finite and solid, and let $s \neq t$ be points of V. Then* `decideHamPath` V s t = `true` *if and only if there is an edge set H of the grid graph on V with degree 1 at s and t, degree 2 at every other point, and one connected component. The cost function* `decideHamPathCost` V s t *is at most* $5\,220\,000\,(|V|+3)^6$.

An edge set with these degrees and one component is exactly a Hamiltonian s–t path. The Lean proof uses only the axioms `propext`, `Classical.choice` and `Quot.sound`.

Cost model. Running time is stated through explicit cost functions written beside each algorithm in Lean. Each test is charged by the cost of its evaluation, and each loop is charged for every iteration. Primitive operations on finite sets (membership, insertion, a cell flip) are charged a constant. The bound is proved for these functions, not for a machine model. The correspondence between an algorithm and its cost function is checked by reading the two definitions side by side. The constant 5 220 000 is 208 final tests of the deletion walks (Section 11) times the constant 25 000 of one final

test, plus the walks' own work. Earlier versions had $5^{12} \cdot 100100 \approx 2.4 \cdot 10^{13}$ (a branching factor of 5 per deletion level) and, with king distance 3 between the out-vertices, $5^{27} \cdot 25100 \approx 1.9 \cdot 10^{23}$.

Prior work in brief. Itai, Papadimitriou and Szwarcfiter posed the solid case [7]. The Hamiltonian path problem in solid grid graphs is listed as open by Keshavarz-Kohjerdi [9] and by Keshavarz-Kohjerdi and Bagheri [11]. The introduction of Sheffield's paper [17] contains the sentence "Similar results hold for Hamiltonian paths with fixed endpoints and for other gridlike families of graphs embedded in n-holed tori." The present work gives a written proof of the path case and checks it in Lean. Appendix A discusses prior work and novelty in detail.

Contributions.

1. *Columns*, which complete the count for a stuck phase, and a locally checkable certificate for the "no" answer (Section 4).
2. Phantom endpoints, which move a degree-one endpoint off the vertex set, and a two-phantom core under a distance condition (Section 5).
3. Bounded deletion walks that reach the distance condition (Section 6).
4. For an endpoint with all eight king neighbours present, a hole whose defect is a *vortex*, a slit height, and a potential that rules out closed climbs (Sections 7 and 10).
5. Flip classes relative to the deleted endpoints, with a class search by relaxation (Section 8).
6. A complete Lean 4 formalization of correctness and a polynomial bound (Section 12).

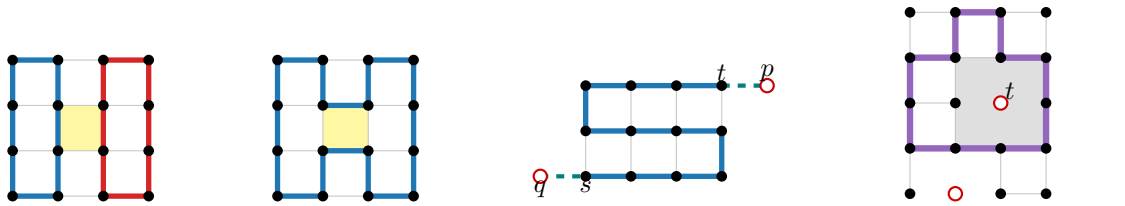
2 Overview

This section explains how the algorithm is put together and why each part is there, before any definitions. Terms are used informally here and defined in the later sections given in parentheses.

The guiding principle. Sheffield's algorithm decides Hamiltonian *cycles* in solid grid graphs [17]. It starts from any 2-factor, a set of edges with degree 2 at every point, which splits V into disjoint cycles called pieces. It then flips unit cells, toggling their four sides, to join pieces one at a time. A height function on the cells orders these flips so that the search cannot run in circles. When no more pieces can be joined, a locally checkable certificate proves that no 2-factor has fewer pieces, so no Hamiltonian cycle exists. In a solid grid graph every 2-factor can be reached from every other by flips, so one run decides the question (Section 4). Figure 2a,b shows one such flip.

Our algorithm reduces the path problem to situations where this machinery still works. Every component below exists for one of two reasons. Either it restores a hypothesis of Sheffield's argument that a path endpoint breaks, or it pays for that repair with a bounded amount of branching. A reader who keeps the cycle algorithm in mind can read each component as an answer to the question "what did the endpoint break, and how is it put back".

Obstacle 1, endpoints have degree one. A Hamiltonian path has degree 1 at s and t. Sheffield's local rules and his height function both use degree 2 at every point. The repair is a *phantom* (Section 5). If an endpoint has a missing lattice neighbour p, the edge from the endpoint to p is added. The endpoint then has degree 2, and the degree-one defect sits at p, outside V, where no cell is ever flipped (Fig. 2c). With phantoms at both endpoints, Sheffield's phase and certificate work



(a) two pieces (b) one flip, one piece (c) phantoms (d) a loop around a hole

Figure 2: The ideas of the algorithm in four pictures. (a) A 2-factor of the 4×4 block with two pieces (blue, red). The yellow cell has its left and right sides in the factor and its top and bottom sides out, and its corners lie in two pieces. (b) Flipping it (toggling its four sides) joins the two pieces into one cycle. Sheffield's algorithm repeats such flips, ordered by a height function. (c) A path from s to t with a phantom edge at each endpoint (dashed) to a missing point p or q (red circles). Now s and t have degree 2, as in a cycle. (d) An inner endpoint t deleted, leaving a hole (grey cells). Two path factors of this leaf differ by the purple cycle, which goes around the hole. No flip changes how many such cycles go around the hole, so the two factors lie in different classes (Section 8, Fig. 13).

again, provided the two endpoints are at king distance at least 2 and their phantoms do not point away from each other across a gap of 2 (the conditions *Far* and *not opposed*). Most of Section 5 is the case analysis that proves this, and the reader can take Theorem 5.5 as its summary.

Getting the endpoints into position. An endpoint may have no usable phantom, because it has no missing neighbour, or because the endpoints are too close or opposed. Then the algorithm deletes the endpoint and branches over its neighbour w on the path. V has a Hamiltonian s–t path exactly when $V \setminus \{t\}$ has one from s to some neighbour w of t, and the deleted t becomes the phantom of w (Lemma 6.1). Repeating this is a *deletion walk* (Sections 6 and 9). It is bookkeeping for correctness. It takes at most 16 starts and at most 13 final tests per start, a constant, and when s and t are far apart it takes no steps at all (Section 14).

Obstacle 2, an inner endpoint leaves a hole. An endpoint whose eight king neighbours all lie in V is *inner*. Deleting it leaves a one-point hole, and the path factor has degree 1 at the hole's centre. Around this point the arcs do not balance, which we call a *vortex*, and no height function exists (Section 7). Two things are lost, and each is put back separately.

- *The order of the flips.* The height is replaced by a *slit height* that jumps across one ray from the vortex, plus a *quadrant label*. Their combination still rises along every climb, which is the only property the certificate uses (Theorem 7.6).
- *Reachability by flips.* Fix one factor F of the leaf as a reference. Any other factor G differs from F by a set of alternating cycles, cycles whose edges lie alternately in F and in G. Count the alternating cycles that go around the hole, with a sign for their direction. This number is the *class* of G (Section 8, with an informal picture at its start). A flip of G changes the difference by the four sides of one cell. The four cells around the hole are not cells of the graph, so no flip changes how many cycles go around the hole, and the class never changes under flips (Fig. 2d). Factors in different classes cannot be reached from each other by flips, so one run of Sheffield's loop no longer decides the question. The algorithm therefore enumerates the classes, at most $2|U| + 1$ per hole, builds the least factor of each class by a relaxation (`classSearch`), and runs the loop in each class. Within one class Sheffield's argument works

component	why it is there	cost	kind	where
loop (phase, reduction, certificate)	Sheffield's cycle algorithm, joins pieces or proves none can be joined	$O(N^3)$ per round, $\leq N$ rounds	core	Section 4, Algorithms 8 and 9
columns	complete the count for a stuck phase	none at run time	proof	Section 4.5
phantoms, Far, not opposed	give endpoints degree 2 so the loop applies	constant	core	Section 5
deletion walk	bring endpoints into the Far, not opposed position	$\leq 16 \cdot 13$ final tests	bookkeeping	Sections 6 and 9, Algorithms 3 and 5
slit height, quadrant label	replace the height at a vortex	none at run time	core (proof)	Section 7
classes, class search	flips do not connect factors around a hole	$(2N)^k$ classes, $O(N^4)$ each	core	Section 8, Algorithm 7
two vortices	both endpoints inner	as above, $k = 2$	core (proof)	Section 10

Table 1: The components of the algorithm. $N = |V| + 3$, and k is the number of inner endpoints. "Core" components carry the argument, "bookkeeping" components make it apply to every position of the endpoints, and "proof" marks components that change the correctness proof but not the computation.

as before. Figure 13 shows a leaf with two nonempty classes.

The class enumeration is the main source of the polynomial degree. Two inner endpoints give two holes and $(2|U| + 1)^2$ class pairs (Section 10).

The loop. For each leaf of the deletion branching and each nonempty class, the algorithm runs Sheffield's loop on two-phantom factors (Section 4). Each round is a phase and a reduction. A phase joins pieces by flipping a merging cell, and when none exists it builds *bridges*, flips that split a piece to expose a merging cell. The reduction then undoes or lowers the bridges, and the round ends with one piece fewer. If a phase is stuck, the certificate proves that no factor of the class has fewer pieces, and the class is rejected. The answer is yes exactly when some leaf and some class reach a single piece. The count behind a stuck phase uses columns (Section 4.5).

What is essential and what is bookkeeping. Table 1 lists the components. The essential ideas are the phantom, the vortex potential, and the classes. The rest makes them applicable everywhere, at a constant or polynomial price.

Names. The procedures have the same names in the text, in the pseudocode of Section 11 and in Lean, written `decideHamPath` in Lean and DecideHamPath in the pseudocode. Table 2 lists them. Lemma names extend the procedure names, for example `decideHamPath_main` and `boundaryEndpointsWalk_correct`.

How to read the paper. A first reading can follow two concrete runs. Section 13.1 traces a small instance with one inner endpoint, where one leaf fails with a certificate and the next succeeds. Section 14 traces an instance with two inner endpoints far apart, where the loop needs a bridge. The pseudocode in Section 11 gives the whole algorithm in one place. After that, Section 4 (the

procedure	what it does	pseudocode
`decideHamPath`	dispatch on which endpoints are inner	Algorithm 1
`boundaryEndpoints`, `boundaryEndpointsWalk`	no endpoint inner	Algorithms 2 and 3
`holeCase`, `holeCaseWalk`	delete t and s, then walk	Algorithms 4 and 5
`oneHoleCase`, `twoHoleCase`	one or two inner endpoints	Algorithm 4
`oneHoleFinalTest`, `twoHoleFinalTest`	the test at a leaf, class by class	Algorithm 6
`classSearch`, `classSearchOne`	the least factor of a class, for several holes or one	Algorithm 7
`twoPhantomTest`, `twoPhantomLoop`	Sheffield's loop with two phantoms	Algorithm 8
`runPhase`	one phase	Algorithm 9
`reducePieces`	flip, then undo or lower the bridges	Algorithm 10
`findFactor`	some factor with the given degrees	Algorithm 11

Table 2: The procedures of the algorithm. The same names are used in the text, in the pseudocode (in small capitals) and in Lean.

loop), Section 7 and Section 8 (holes) carry the main argument, and Sections 5, 6 and 9 can be read for the endpoint details as needed.

3 Preliminaries

3.1 Grid graphs, cells and solidity

A *point* is an element of $\mathbb{Z}^2$, written (x, y). *Right* means increasing x and *up* means increasing y. The *king distance* of two points is $d_\infty(p, p') = \max(|x - x'|, |y - y'|)$. The *king neighbours* of p are the eight points at king distance 1. The *lattice neighbours* of p are the four points at Euclidean distance 1.

Definition 3.1 (grid graph, edge, cell)**.** Let $V \subset \mathbb{Z}^2$ be finite. An *edge* of V is a unit segment whose two ends lie in V (Lean `IsEdge`). The *grid graph* on V has these edges. A *square* is a unit square with lattice corners. It is named by its lower-left corner c, so the square $c = (x, y)$ has corners (x, y), $(x + 1, y)$, $(x, y + 1)$, $(x + 1, y + 1)$. A square is a *cell* of V when all four corners lie in V (Lean `IsCell`). The four edges of a square are its *sides*, named bottom, top, left and right.

Definition 3.2 (solid)**.** A point $a \notin V$ is a *missing point*. V is *solid* when every missing point is joined, by a sequence of king moves through missing points, to a point strictly left of all of V (Lean `Solid`, `Grid.lean`).

Figure 3 shows a solid set and a set that is not solid. A bounded face of area greater than one contains a missing point that cannot escape in this way.

Definition 3.3 (inner point)**.** A point p is *inner* in V when the four squares with corner p are cells of V (Lean `Inner`, decided by `isInner`). Equivalently, all eight king neighbours of p lie in V. A point of V that is not inner has a missing king neighbour (`miss_of_not_inner`).

3.2 Factors and pieces

Definition 3.4 (factor, path factor, piece)**.** Let $d\colon \mathbb{Z}^2 \to \mathbb{N}$. A *factor* of V with degrees d is a set H of edges of V in which every $p \in V$ has degree $d(p)$ (Lean `IsFactor`). For $s \neq t$ let d_{st} be 1 at s and

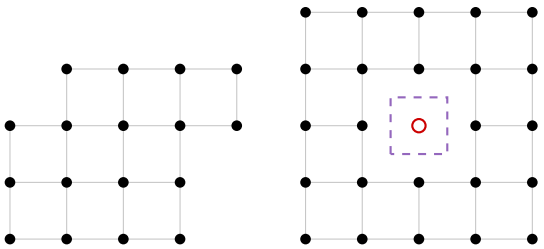


Figure 3: Left: a solid grid graph. Right: not solid. The red point is missing. All eight of its king neighbours (inside the purple dashed box) are in V, so it cannot reach the outside by king moves through missing points.

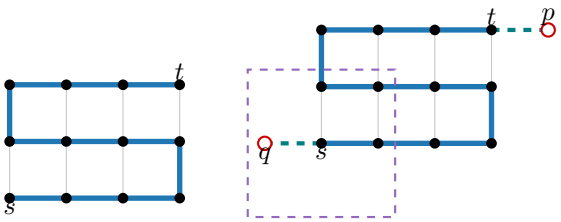


Figure 4: Left: a path factor with one piece, a Hamiltonian path from s to t. Right: the same path with two phantom edges (dashed, Section 5). The missing points p and q are phantoms, and t and s now have degree 2. The purple box holds the points within king distance 1 of s. The point t lies outside it, so s and t are Far (Definition 5.3). Their phantoms are not opposed, because t and s differ by 3 in x.

t and 2 elsewhere (Lean `dST`). A factor with degrees d_{st} is a *path factor*. The connected components of H on V are its *pieces*, and $\mathrm{ncomp}(V, H)$ is their number (Lean `ncomp`).

A path factor consists of one path from s to t and some cycles. It is a Hamiltonian s–t path exactly when it has one piece (Fig. 4, left). An edge of H is called *dark*. An edge of V not in H is called *light*.

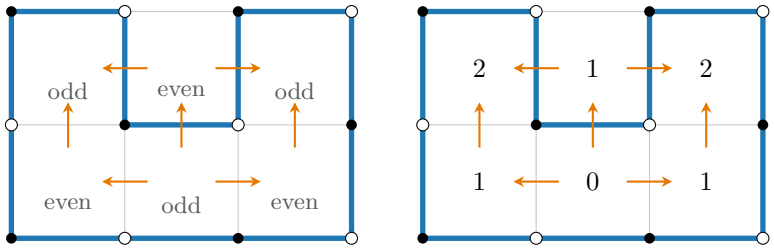
Finding a factor. Every run starts from some factor with prescribed degrees $d \leq 2$. The grid graph is bipartite, so such a factor is an integral maximum flow in the standard source–black–white–sink network, with capacity d at the points and 1 on the edges. `findFactor` (`FindFactor.lean`, Algorithm 11) runs the Ford–Fulkerson method on this network, with at most $|V|+1$ augmentations and every choice fixed. It returns a factor when one exists (`findFactor_some`), and returns none only when no factor exists (`findFactor_none`), at cost $O((|V|+1)^3)$ (`DFC.findFactorCost_le`). The factor usually has several pieces. Appendix E gives the network, the colour conditions that degree 1 at s and t imposes, and two examples.

3.3 Arcs and heights

The *colour* of a square $c = (x, y)$ is the parity of $x + y$ (Lean `par`). Two squares that share a side have different colours.

Definition 3.5 (arc)**.** Let H be a set of edges and let c be a square with a side e. The *arc* of H across e points out of c exactly in the following cases (Lean `outArc`, closed form `outArc_eq`).

- e is horizontal, and either c is odd and e is dark, or c is even and e is light.



(a) arcs (b) a height τ

Figure 5: (a) A Hamiltonian cycle (blue) on a 4×3 block, the colour of each cell, the checkerboard colour of each point (black or white), and the arc across each side between two cells (orange arrows). Walking along an edge from its black end to its white end, the arc of a dark edge points left and the arc of a light edge points right. The side between the even cell $(0, 0)$ and the odd cell $(1, 0)$ is vertical and light, so by Definition 3.5 its arc points into $(0, 0)$. (b) The same arcs with a height τ written in each cell, normalised so the least value is 0. Every arrow goes from a cell to a cell whose height is larger by 1.

- e is vertical, and either c is odd and e is light, or c is even and e is dark.

Otherwise the arc points into c.

Colour the points like a checkerboard, a point (x, y) *black* when $x+y$ is even and *white* otherwise (Lean `Black`). Every edge joins a black point to a white point. The colour of the square on each side of an edge is fixed by which end of the edge is black, so Definition 3.5 can be restated without cell colours. Walk along an edge from its black end to its white end. If the edge is dark, its arc crosses it from right to left. If the edge is light, its arc crosses it from left to right. This agrees with `outArc` on every edge (checked on all edges of random edge sets on a 5×5 block).

The rule is antisymmetric. Seen from the square on the other side of e (of the other colour), the same arc points in. So each side between two squares carries one arc, pointing from one square to the other. Toggling the membership of e reverses its arc.

Definition 3.6 (height)**.** A *height* of H on the cells of V is a function τ on cells with $\tau(c') = \tau(c) + 1$ whenever the arc across the side between cells c and c' points from c to c' (Lean `HeightOn`).

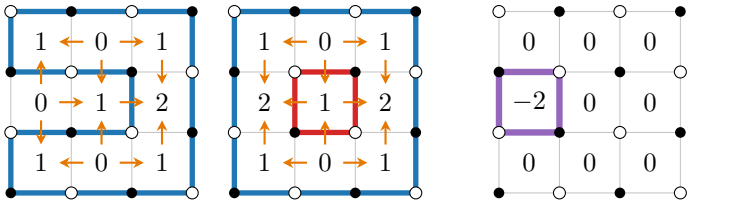
Figure 5 shows the arcs of a Hamiltonian cycle on a 4×3 block and its height. Each arc is drawn as a short arrow across the side it belongs to. The height rises by 1 along every arrow.

Around a point v, the four squares with corner v are separated by the four edges at v. Going once around v, the arcs add up to $\pm(4 - 2\deg_H v)$, with the sign fixed by the colour of v. So the arcs balance at a point of degree 2.

Lemma 3.7 (`height_exists`, `Height.lean`)**.** *If V is solid and every inner point has degree 2 in H, then H has a height on the cells of V.*

Definition 3.8 (relative height D)**.** Let F and H be edge sets. Their *symmetric difference* is $F \oplus H = (F \setminus H) \cup (H \setminus F)$, the set of edges that lie in exactly one of F and H. Give each edge e of $F \oplus H$ the weight $+2$ if the arc of F across e points from its lower or left square to its upper or right square, and -2 otherwise. $D(F, H)(c)$ is the sum of these weights over the vertical edges of $F \oplus H$ in the row of c at or left of the left side of c (Lean `D`, `D.lean`).

When F and H have the same degrees at every point, $D(F, H)$ changes by ± 2 across each side in $F \oplus H$ and by 0 across every other side. Its sign across $e \in F \oplus H$ is $+$ in the direction of the arc



(a) F and τ_F (b) H and τ_H (c) $F \oplus H$ and $D(F, H)$

Figure 6: (a) A Hamiltonian cycle F on a 4×4 block, its arcs and its height τ_F. (b) The factor H obtained by flipping the alternating cell $(0, 1)$ of F, with two pieces (blue, red), its arcs and its height τ_H. (c) The symmetric difference $F \oplus H$ (purple), the four sides of $(0, 1)$, and $D(F, H)$ in each cell. The only nonzero value is -2 at $(0, 1)$, which equals $\tau_F - \tau_H$ there.

of F. When both factors have heights, $D(F, H) = \tau_F - \tau_H$ up to a constant. D needs no boundary values, and it is defined for path factors as well. Figure 6 shows two factors that differ at one cell.

Lemma 3.9 (properties of D)**.** *Let F, H be factors with the same degrees.*

1. *$D(F, H)$ is even (`D_even`).*
2. *$D(F, H)$ never decreases along an arc of F (`D_mono_arc`, `D_step`).*
3. *If V is solid and $F \oplus H$ consists of edges of V, then $D(F, H)(c) = 0$ for every square c that is not a cell of V (`D_eq_zero_of_not_cellW`).*

The conclusion of part 3 is named `SolidFor` V F H. Large parts of the argument use only `SolidFor`, and not solidity itself. This is what makes holes tractable in Section 8.

3.4 Flips and cut cells

Definition 3.10 (alternating cell, flip)**.** A cell is *alternating* in H when two opposite sides are dark and the other two are light (Lean `Alt`). The *flip* of a cell toggles its four sides (Lean `flip`).

A cell is alternating exactly when its four arcs all point out or all point in (`alt_iff_uniform`). So an alternating cell is a local minimum or maximum of the height. Flipping it keeps every degree, reverses its four arcs, and changes its height by ± 2.

Definition 3.11 (cut cell, merging cell)**.** A cell is a *cut cell* of H when its four corners do not all lie in one piece (Lean `CutSq`). A *merging cell* is an alternating cut cell.

Flipping a merging cell joins the two pieces at its two dark sides, so the number of pieces drops by one. Flipping an alternating cell that is not cut splits a piece or keeps the count. Figure 7 shows a merging flip.

4 Sheffield's phase and the certificate

This section describes the cycle algorithm of [17, §4] in the form used in Lean. In the Lean development a factor lives on a set $W \supseteq V$. Only cells of V are flipped. Every corner of a cell of V has degree 2 (in Section 5 this is arranged for path factors).

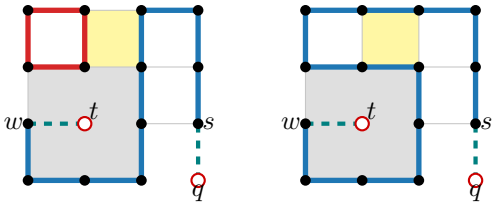
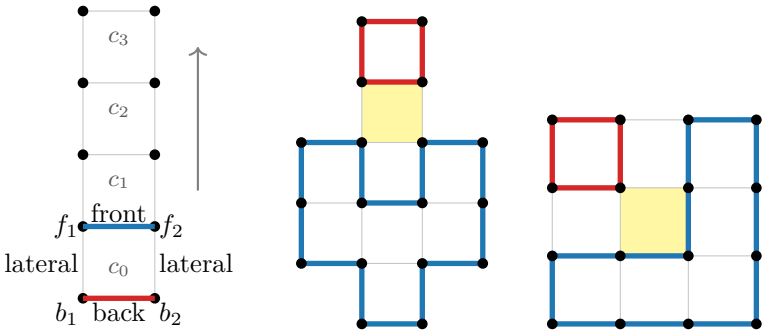


Figure 7: A flip joining two pieces, from the 4×4 instance of Section 13.2. Left: a factor with two pieces, a path (blue) and a 4-cycle (red). The yellow cell is alternating and cut, so it is a merging cell. Right: after the flip, one piece. Grey squares are the four hole cells around the deleted inner endpoint t (Section 7). Dashed edges are phantom edges.



(a) a frame (b) a critical cell (c) a corner cell

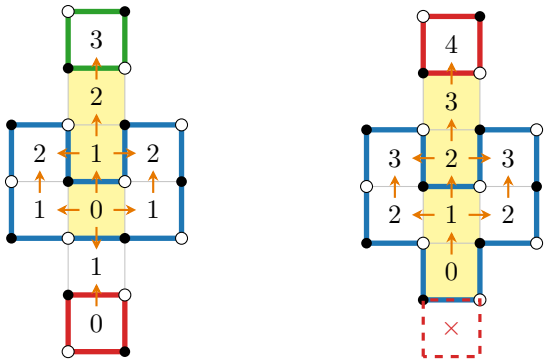
Figure 8: (a) A frame of the cell c_0 with its front up, the corners b_1, b_2, f_1, f_2, and the strip c_0, c_1, c_2, c_3 in the direction of the front. (b) A critical cell (yellow) in a plus-shaped set with a factor of two pieces (blue, red). Its back, the top side, is dark and lies on the red 4-cycle. Its front, the bottom side, is light, and its front corners lie on the blue cycle. (c) A corner cell (yellow) in a 4×4 block. Its top-left corner is alone in its piece among the four corners. The two sides at the opposite corner, the bottom-right one, are dark.

4.1 Frames, critical cells and rows

Definition 4.1 (frame)**.** A *frame* of a cell c is a choice of one of its sides as the *front*. The opposite side is the *back*, and the other two are the *laterals*. The corners on the back are b_1, b_2 and those on the front are f_1, f_2, with laterals $b_1 f_1$ and $b_2 f_2$ (Lean `back`, `front`, `lat1`, `lat2`, `back1`, `front1`, `Strip.lean`). The j-th cell of the strip from c in the direction of its front is c_j, with $c_0 = c$ (Lean `cellJ`).

Definition 4.2 (critical cell, corner cell)**.** A cell c is *critical* in a frame when b_1, b_2 lie in one piece, f_1, f_2 lie in one piece, these two pieces differ, and exactly one of back and front is dark (Lean `CriticalIn`). A cell is a *corner cell* when one corner is alone in its piece among the four corners, the other three share a piece, and the two sides at the corner opposite the lone corner are dark (Lean `CornerIn`).

Sheffield's Lemma 1 says the following. If no cut cell is alternating, then every cut cell is critical or a corner cell, and no cell touches three pieces. The proof uses degree 2 at the corners. In Lean the cut-cell shapes are in `Shapes.lean` and `E1.lean`. At a degree-one point near the cells the lemma has exceptions. One of them, called E1, touches three pieces (Section 5).



(a) a row of length 3 (b) a stuck cell, heights drawn

Figure 9: Rows in plus-shaped sets, with the arcs (orange) and a height in every cell. (a) A factor with three pieces. The cell $(1, 4)$ is critical in the frame whose front is its bottom side. Its row (yellow) is $(1, 4), (1, 3), (1, 2)$, of length 3. The heights fall $2, 1, 0$ along it, so the arc across the front of each of its cells points up. The arc across the front of $(1, 2)$ points down, from height 0 to height 1, so the row ends at $(1, 2)$. (b) The same shape without its two bottom points, and a factor with two pieces. The row from $(1, 4)$ falls $3, 2, 1, 0$ through four cells (yellow), and the next square, $(1, 0)$ (red cross), is not a cell of V. So $(1, 4)$ is stuck. It is the only critical cell, the factor has no merging cell, and the phase ends stuck. By the certificate (Theorem 4.8) this set has no Hamiltonian cycle.

Definition 4.3 (row, stuck)**.** Let q be critical in a frame with dark back. The *row* from q runs through $q = c_0, c_1, \ldots$ while the arc across the front of each cell points the same way as at q. Its last cell c_{r-1} is the first one whose front arc points the other way. The row is *well formed* when $c_1, \ldots, c_{r-1}$ are cells of V and are not cut. The cell q is *stuck* when the run reaches a square c_k ($k \geq 1$) that is not a cell or is a cut cell before its arc turns (Lean `StuckAt`, `ColProof.lean`).

The heights rise strictly along the row (or fall strictly, depending on the direction of the arcs). The last cell is a local maximum (or minimum) along the row. Figure 9 shows a row of odd length and a stuck cell, with the heights drawn in.

Definition 4.4 (bridge, fixed pair)**.** A *bridge step* applies to a well formed row of even length $r \geq 2$ from a critical cell q that is not a square of an earlier bridge (Lean `RowStep`, `Bridges.lean`). It flips only the last cell $E = c_{r-1}$, and it requires that this flip splits a piece (`RowStep.split`). The two last cells $(A, E) = (c_{r-2}, c_{r-1})$ form the *fixed pair* of the bridge (Lean `FixedPr`, `Bridge.pairOf`).

4.2 The phase

A *phase* starts from a factor H_0 with m pieces and no bridges. It repeats the following test in a state H with a list of bridges (Lean `phase`, `SPhaseAlg.lean`).

1. If some merging cell is not a square of a fixed pair, the phase ends with outcome *merge*.
2. Otherwise it looks for a critical cell with dark back that is not stuck and not a bridge square. If none exists, the phase ends with outcome *stuck*.
3. If that cell's row has odd length, the phase ends with outcome *odd*.
4. If the row has even length and flipping its last cell splits a piece, the phase makes a bridge step and continues.

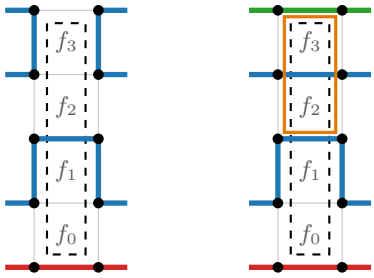


Figure 10: An even row of length $r = 4$ upward from a critical cell f_0 (dashed box), and the bridge made from it. The window shows the edges forced by the bridge pattern (`RegionOK`). Short stubs are edges that leave the window. Left: before the step, f_0 has a dark back (red piece below) and the strip pattern above it (blue piece). The top cell f_3 has dark laterals and light back and front. Right: f_3 is flipped. Its back and front are now dark. The strand through its top corners has split off from the blue piece (green). Which of the two strands keeps the blue piece depends on edges outside the window, and the colouring shows one possibility. The orange box marks the fixed pair (f_2, f_3).

5. If the row has even length and flipping its last cell does not split, the phase ends with outcome *twist*.

Each bridge step adds one piece, so a state reached after k steps has $m + k$ pieces (`reach_count`, `PhaseCount.lean`). The case split is exhaustive (`phase_spec`).

4.3 Success branches and the undoing of bridges

The outcomes merge, odd and twist each give a *half bridge*. The half bridge is a flip of a merging cell, the lowering of an odd row, or the lowering of an even row. Lowering a row flips its cells from the last down to the first. Each of these operations joins one pair of pieces. The bridges are then processed in reverse order of creation. Bridge i is *lowered* (its row without E_i is flipped) when the pending pair of points is connected in the state before bridge i. Otherwise it is *undone* (E_i is flipped back). This rule is computable (Lean `plan`, `planB`). It reproduces Sheffield's path in his boundary tree without building the tree.

Theorem 4.5 (Theorem I.8, `undo_count`, `UndoCount.lean`)**.** *If the operations of the plan are compatible, then applying the half bridge and the plan to the final state of the phase gives a bridge-free factor with $m - 1$ pieces.*

Compatible (`OpsOK`) means two things. The cell sets of different operations are disjoint. No edge toggled by an earlier operation is a side of a cell of a later one. This is Sheffield's Lemma 4. It is proved in Lean as Theorem II for every operation pair (`opsOK_rows`, `opsOK_merge`). One configuration of the merge branch, two bridge ends crossed at one merging cell, is not excluded. It is avoided by a choice rule. A merging cell X is *good* when no bridge i ends at X (the square beyond E_i is X) while i is not on the chain of lowered bridges.

Theorem 4.6 (Theorem R, `theoremR`, `S4TheoremR.lean`)**.** *In the merge outcome a good merging cell exists.*

The algorithm takes the first cell passing the computable test `goodB`. It is computed as `isGoodMergingCell`, which builds the connections of each intermediate state once (`isGoodMergingCell_eq`).

4.4 The certificate

Definition 4.7 (certificate, `Cert`, `Cert.lean`)**.** Let H be a factor, FP a list of fixed pairs, and t_0, u_0 points (the out-vertices of Section 5). `Cert` V H FP t_0 u_0 is the conjunction of the following local conditions.

(1) Every merging cell is a square of a pair in FP (`f2`).

(2) Every pair satisfies `PairOK`. The light sides of the top square have arcs pointing in, those of the bottom square point out, the arc from bottom to top points up, and the corner pieces are recorded.

(3) Every cell touching three or more pieces has the E1 shape at t_0 or u_0 or the diagonal shape (`TriOK`).

(4) Every edge with one end outside V has its other end at t_0 or u_0 (`out`).

(5) No cell with joined back corners, joined front corners and different back and front pieces has both back and front light (`l4`).

(6) Pair squares are alternating, the shared side of a pair is dark, and pairs are disjoint.

(7) For every critical cell with dark back that is not a pair square, its *column* (Section 4.5) ends at a pair square of its direction, or it leaves V with all its arcs pointing one way (`col`).

(8) The same holds for strips that start beside the square just beyond a pair (`side`).

`Cert` mentions only V, H, FP, t_0 and u_0. It does not mention any competitor factor.

Theorem 4.8 (`cert_count_gen`)**.** *Let H and H' be factors of $W \supseteq V$ with the same degrees, agreeing off the edges of V. Assume every corner of a cell of V has degree 2, `SolidFor` V H H' holds, and no factor of W with these degrees has a closed climb (`ClimbAcyc`, Definition 4.14). If `Cert` V H FP t_0 u_0 holds, then* $\mathrm{ncomp}(W, H) \le \mathrm{ncomp}(W, H') + |\mathrm{FP}|$.

Theorem 4.9 (`cert_of_stuck`, `TriProof.lean`)**.** *Let the phase start from a factor whose edges leaving V all end at t_0 or u_0, and let t_0, u_0 be Far (Definition 5.3). If the phase ends with outcome stuck in state H with bridge list* BS*, then `Cert` V H* (BS.map `pairOf`) t_0 u_0 *holds.*

A stuck state has $m + r$ pieces with $r = |\mathrm{BS}|$. If a Hamiltonian path exists, Theorem 4.8 with H' the path gives $m + r \le 1 + r$. So $m \le 1$, and a phase from $m \ge 2$ pieces never ends stuck. The "no" answer is therefore certified by a local check. It does not depend on the phase rules. The phase rules are needed only for completeness, that is, for showing that a phase ends in a merge or in a certified state.

4.5 Columns and the stuck case

The count behind Theorem 4.8 compares a stuck state H with a competitor H'. Fixed pairs are moved along rows toward H', and the cut components on which $D(H, H')$ is not constant are then counted. A critical cell can be stuck against another cut cell, and the row then continues through several cuts as one straight column. Sheffield's written argument (his §4.8) treats rows only, and Appendix D shows where columns change the count.

Definition 4.10 (strip, column)**.** A *strip* of length n from c is a straight run $c_0, \ldots, c_{n-1}$ with dark back of c_0, laterals of c_j dark exactly for odd j, and the front of c_j dark exactly for odd j, except that the last front is dark exactly when n is odd (Lean `StripPat`). A *column* is a strip through critical cells and plain strip cells, ending at a fixed-pair square (Lean `ColEnds`).

Lemma 4.11 (column lemma, `ColProof.lean`)**.** *Let f be critical, and let its row run into a cut cell g that is higher than the cell before it. Then g is critical, extremal along its cut, and its row continues in the same direction.*

Lemma 4.12 (strand pairing, `lowerCol_count`, `Column.lean`)**.** *Lowering a column toggles all its laterals and its two end sides. It keeps the pairing of the strands that enter and leave the column at odd heights. Only the two ends of the column change pieces. So a column changes the number of pieces exactly as one row of the same length.*

The proof uses columns in place of rows in the move. A *generalized move* at a critical cell c with D of its direction's sign lowers the column from c_1 to its top and makes (c_0, c_1) the new fixed pair. It keeps the number of pieces (Lemma 4.12), keeps the invariant `Inv` (`MoveInv.lean`, `ColAfter.lean`, `SideAfter.lean`, `E1After.lean`), and lowers $\mu = \sum_c |D(H, H')(c)|$ by $2(L-1)$ for a column of length L. After all moves, every cut component on which $D \neq 0$ contains a fixed-pair square of the right sign. The count then uses a rank bound that sees only these joins (`ncomp_le_of_joins`, `Rank.lean`). The square beyond a pair created by a move need not be a cut cell, and the invariant has a field `side` for it. The E1 square is handled by a shape invariant instead of a count.

Theorem 4.13 (`stuck_minimal`, `Minimal.lean`)**.** *If* `Inv` V H H' FP t_0 u_0 *holds, then* $\mathrm{ncomp}(W, H) \leq \mathrm{ncomp}(W, H') + |\mathrm{FP}|$.

The proof is an induction on μ. `Cert` implies `Inv` for every competitor (`cert_inv`). A column that leaves V cannot carry D of its direction's sign, because D grows along arcs and vanishes off the cells (`not_colOut`).

4.6 Climbs

Definition 4.14 (climb)**.** A *climb step* in direction o goes from a cell c to a neighbouring cell c' across a side whose arc points from c to c' (for $o =$ `true`) or from c' to c (for $o =$ `false`), and which joins two given pieces (Lean `ClimbStep`). `ClimbAcyc` says that no sequence of climb steps returns to its start. `ArcAcyc` is the same condition without the piece restriction (`SlitHeight.lean`).

The final count follows a climb from a cell with $D \neq 0$ along its cut to an extremal cell (`climb`, `Final.lean`). With a height the climb ends because τ rises (`acyc_of_height`). The Lean proof of `climb` uses only `ClimbAcyc`. Its measure is the number of cells reachable by climb steps.

4.7 The loop

The loop `runSC` starts from any path factor. It repeats phases, and each successful phase is followed by the reduction of Theorem 4.5. Each round removes one piece. The loop answers `true` when one piece remains and `false` on a stuck phase. Its fuel is $|V| + 2$ rounds, and each phase has fuel $|W| + 1$ bridge steps.

The algorithms use a fast version of the two-phantom loop, `twoPhantomLoop`, with the same answers.

- `connectedPairs` W H lists all connected pairs of points in one sweep, computing each component once, in $O(|W|^2)$ steps (`connectedPairsCost_le`, `isConnected_eq`, `ConnSet.lean`).
- The phase `runPhase` computes the connection set once per step, together with the set of bridge cells, so each step costs $O(N^2)$. It equals `phase` on the states the phase reaches (`runPhase_eq`, `PhaseS.lean`).

- The reduction `reducePieces` computes the connection sets of the bridge states once, and the plan of each candidate merging cell once, in $O(N^3)$ steps (`reducePieces_eq`, `ReduceS.lean`).
- `twoPhantomLoop` equals `runSC2` (`twoPhantomLoop_eq`) and costs at most $(n{+}1)\cdot 3000\,(|V|{+}3)^3$ for fuel n (`twoPhantomLoopCost_le`, `LoopF.lean`).

Here $N = |W| + 1$.

5 Phantom endpoints and the two-phantom core

Sheffield's argument needs degree 2 at every corner of a flipped cell. A path endpoint has degree 1. The fix is to move each endpoint off V.

Definition 5.1 (phantom)**.** Let $t \in V$ have a missing lattice neighbour p. A *phantom factor* is a factor on $V \cup \{p\}$ with degrees d_{sp} that contains the edge t–p and no other edge at p (Lean `NotchF`, `NotchPF.lean`). The point p is the *phantom* of t, and t–p is the *phantom edge*.

In a phantom factor t has degree 2 and the endpoint is $p \notin V$. Phantom factors correspond one to one to path factors of V (`notch_up`, `notch_down`), with the same pieces. Only cells of V are flipped, so p is never the corner of a flipped cell. A point is *dangling* when it is the corner of no cell. If the other endpoint s is dangling, every corner of a cell of V has degree 2.

Definition 5.2 (two phantoms)**.** Let t and s have missing lattice neighbours $p \neq q$. A *two-phantom factor* is a factor on $V \cup \{p, q\}$ with degree 1 at p and q and 2 on V, containing t–p and s–q and no other edge at p or q (Lean `NotchF2`, `TwoPh.lean`).

Definition 5.3 (out-vertex, Far)**.** An *out-vertex* of a factor is a point of V joined by an edge of the factor to a point outside V. In a two-phantom factor the out-vertices are t and s (`two_out`). Two points t_0, u_0 are *Far* when $t_0 = u_0$ or $d_\infty(t_0, u_0) \geq 2$ (Lean `Far`, `Close1`, `StuckL4.lean`).

Definition 5.4 (opposed phantoms)**.** Let a, b be out-vertices with phantoms m_a, m_b. The phantoms are *opposed* when a and b differ by exactly 2 along one axis and by at most 2 along the other, and m_a and m_b point away from each other along the first axis (Lean `Opp`). For example $a = (0,0)$, $m_a = (-1,0)$, $b = (2,1)$, $m_b = (3,1)$ are opposed. `NoOpp` V H t_0 u_0 says that no two distinct out-vertices of H carry opposed phantoms. For a two-phantom factor, `FarN` t p s q means that t, s are Far and the phantoms p of t and q of s are not opposed (`TwoPh.lean`). `FarN` gives `NoOpp` for every two-phantom factor (`two_noOpp`).

The certificate argument was first written with one out-vertex. Every place that used uniqueness was found by the Lean compiler when the second out-vertex was added (`l4_of`, `tri_of`, `a2p`, `side_or_t`, `t_farA`, and others). Almost every site closes under Far, because two different corners of one cell lie within king distance 1 (`two_sides`, `corners_close`). Two sites need more, `t_farA2` and `t_farB2` in the proof of Theorem R (its Lemma 2.3, the crossed configuration). There one out-vertex sits at a corner of the bridge end E and the other at a corner of a cell two steps away, at king distance 2 in the worst case. Both sites close by `NoOpp`, using the phantom positions returned by `side_or_t`. So `theoremR` takes the hypotheses Far and `NoOpp` for the start factor of the phase. Without Far the certificate field L4 can be false in a real stuck state. Such states exist with s and t on one grid line at distance 1 or 2 and phantoms pointing away from each other.

Theorem 5.5 (`hpSheffC2_main`, `TwoPhCost.lean`)**.** *Let V be solid, and let $s \neq t$ in V have missing lattice neighbours $q \neq p$ with `FarN` t p s q. Then `hpSheffC2` decides whether V has a Hamiltonian s–t path (`hpSheffC2_main`). Its fast version `twoPhantomTest` returns the same answer (`twoPhantomTest_eq`) within $5000(|V|+3)^4$ steps (`twoPhantomTestCost_le`, `LoopF.lean`).*

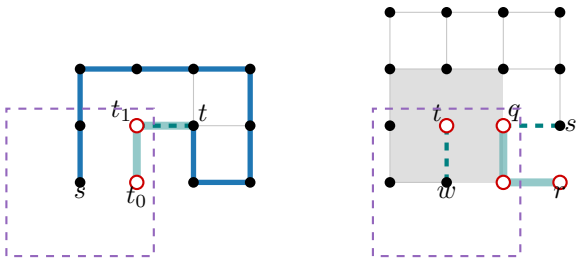


Figure 11: Left: the deletion walk with two boundary endpoints. The Hamiltonian path from s to t_0 ends with t, t_1, t_0, and its deleted part is teal. The walk deletes t_0 and then t_1, because each lies inside the purple box (king distance at most 1 from s). The endpoint t is outside the box, so s, t are Far. The deleted t_1 is a usable phantom of t. Right: the s-side walk in the 4×4 instance of Section 13.2 (Section 9). The inner endpoint t has been deleted (grey hole cells) and w is its neighbour on the path. The deleted path P (teal) runs from r (the original s) to the phantom q. The current endpoint s is joined to q by a phantom edge (dashed). The purple box holds the points within king distance 1 of w.

6 Deletion walks

Lemma 6.1 (deletion, `del_iff`, `TwoPhWalk.lean`)**.** *Let $s \neq t$ be in V with $|V| \geq 3$. Then V has a Hamiltonian s–t path if and only if, for some lattice neighbour $w \neq s$ of t in V, the set $V \setminus \{t\}$ has a Hamiltonian s–w path.*

The deleted point t is a missing lattice neighbour of w, so it can serve as the phantom of w. If t has a missing king neighbour, $V \setminus \{t\}$ is solid (`solid_erase`).

The walk. The walk `boundaryEndpointsWalk` keeps s with its phantom fixed and moves t. In state (V, t) it does the following. A *usable phantom* of t is a missing lattice neighbour of t that differs from the phantom q of s and is not opposed to q (Lean `isUsablePhantom`). If t has a usable phantom and t, s are Far, the walk runs `twoPhantomTest` with that phantom. Otherwise it deletes t and branches over the lattice neighbours $w \neq s$ of t in V. The measure `wmu` is the number of points of $V \setminus \{s\}$ within king distance 1 of s (at most 8), plus 2 if t has no missing lattice neighbour, plus 1 if t has no usable phantom. It is at most 11, and each deletion lowers it. So the walk has at most 11 deletion levels with at most 4 branches each, and its fuel is 12 (Fig. 11, left).

Two boundary endpoints. `boundaryEndpoints` handles s and t that each have a missing king neighbour. If s has a missing lattice neighbour, that is its phantom. Otherwise s is deleted once first (Lemma 6.1 read with the roles of s and t exchanged).

Theorem 6.2 (`boundaryEndpoints_main`)**.** *For solid V and $s \neq t$ in V each with a missing king neighbour, `boundaryEndpoints` decides the problem within $1\,060\,000\,(|V| + 3)^4$ steps.*

7 Inner endpoints: holes, vortices and slit heights

An inner endpoint has no missing king neighbour, so no phantom is available. The algorithm deletes it anyway.

Definition 7.1 (hole)**.** Let t be inner in the solid set V, and let w be the neighbour of t on a Hamiltonian path. Put $V' = V \setminus \{t\}$. The point t is a *hole* of V', because all eight of its king neighbours lie in V'. The four squares with corner t are the *hole cells*. They are cells of V but not of V', so they are never flipped. The point t is the phantom of w, and it lies inside the hole.

Two facts that solidity gave now fail. There is no height (this section), and factors need not be connected by flips (Section 8).

Definition 7.2 (vortex)**.** A *vortex* of H is an inner point of degree 1.

At a vortex the four arcs do not balance. Going once around it, they add up to ± 2 (`wS_defect1`). So no height exists on the cells of V. In a phantom factor for the inner endpoint, t has degree 1 (its only edge is w–t), so t is a vortex.

Definition 7.3 (slit)**.** The *slit* of a vortex (a, b) is the set of horizontal edges on row b strictly left of a (Lean `SlitE`).

Theorem 7.4 (slit height, `slit_height`, `SlitHeight.lean`)**.** *Let V be solid, let (a, b) be inner with $\deg_H(a, b) = 1$, and let every other inner point have degree 2. Then there are $\tau\colon \mathbb{Z}^2 \to \mathbb{Z}$ and $\delta \in \{2, -2\}$ such that for every side e between two cells of V*

$$\tau(c^+) - \tau(c^-) = \varepsilon(e) + \begin{cases} \delta & \text{if } e \text{ is in the slit,} \\ 0 & \text{otherwise.} \end{cases}$$

Here c^- is the lower or left square of e, c^+ the upper or right one, and $\varepsilon(e)$ is $+1$ if the arc of H across e points from c^- to c^+ and -1 otherwise.

The proof routes the imbalance δ as a unit flow from the vortex straight left past all of V, and subtracts it (Appendix C.2).

Definition 7.5 (quadrant label)**.** For a vortex (a, b) and a square $c = (x, y)$, $\mathrm{lab}(c)$ is 0 if $y < b$ and $x < a$, 1 if $y < b$ and $x \geq a$, 2 if $y \geq b$ and $x \geq a$, and 3 if $y \geq b$ and $x < a$ (Lean `lab`).

The four squares around the vortex have labels $0, 1, 2, 3$ counterclockwise from the lower left. Across the vertical line through the vortex and across the ray right of it, the label changes by ± 1. Across the slit it changes by ± 3 (`lab_step`, Fig. 12).

Theorem 7.6 (no closed climb around a vortex, `arc_vortex`)**.** *Under the hypotheses of Theorem 7.4, H has no closed walk of arc steps between cells of V in a fixed direction. In particular `ClimbAcyc` V H holds (`acyc_vortex`).*

Proof. Fix the direction o and put $g = 1$ for $o =$ `true` and $g = -1$ otherwise. Take τ, δ from Theorem 7.4. Put $\mu = -1$ if $g\delta = 2$ and $\mu = 1$ if $g\delta = -2$, and

$$\Phi(c) = 2g\,\tau(c) + \mu\,\mathrm{lab}(c).$$

Consider a step from c to c' whose arc points in direction o. For a function f on squares write $\Delta f = f(c') - f(c)$ for the change of f along this step. By Theorem 7.4, $g(\tau(c') - \tau(c)) = 1 + g\kappa$ where κ is δ for an upward crossing of the slit, $-\delta$ for a downward crossing, and 0 otherwise.

- Away from the slit, $2g\Delta\tau = 2$ and $|\Delta\,\mathrm{lab}| \leq 1$, so $\Delta\Phi \geq 1$.
- Upward across the slit, $\Delta\,\mathrm{lab} = +3$ and $2g\Delta\tau = 2 + 2g\delta$. If $g\delta = 2$ then $\mu = -1$ and $\Delta\Phi = 6 - 3 = 3$. If $g\delta = -2$ then $\mu = 1$ and $\Delta\Phi = -2 + 3 = 1$.

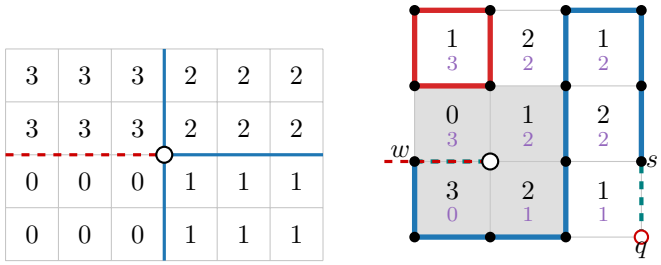


Figure 12: Left: the quadrant label around a vortex (open circle). It changes by ± 1 across the blue lines and by ± 3 across the slit (red dashed). Right: the factor G_0 of the 4×4 instance of Section 13.2. The open circle is the deleted inner endpoint t, a vortex of the factor. Each cell shows its slit height τ (bold, shifted so the least value is 0) and its quadrant label (small, purple). The slit is the single side left of t on its row. The function $2\tau + \text{lab}$ rises along every arc (Section 13.2).

- Downward across the slit, $\Delta \text{lab} = -3$ and $2g\Delta\tau = 2 - 2g\delta$. If $g\delta = 2$ then $\Delta\Phi = -2 + 3 = 1$. If $g\delta = -2$ then $\Delta\Phi = 6 - 3 = 3$.

So Φ rises strictly along every step, and a closed walk would return Φ to its start value. □

No Jordan curve argument is used. `arcAcyc_transfer` passes the conclusion to a subset of the cells and to any factor whose arcs agree on the sides between those cells.

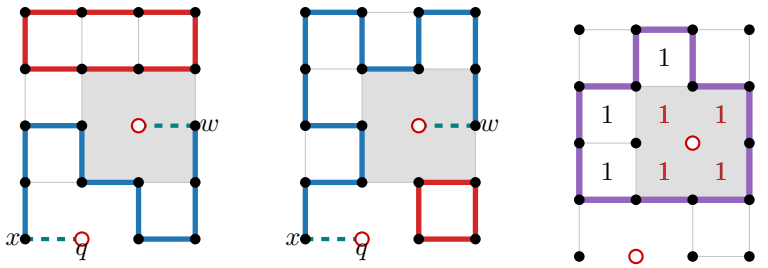
8 Classes and the class search

With a hole, two factors with the same degrees need not be connected by cell flips. Sheffield's Theorem 8 [17, §4.1] indexes the flip components of 2-factors on a grid graph with holes by the relative height on the holes. The classes below follow this idea. They are taken relative to phantom and deleted endpoints, and the degree-one defect is handled by Section 7.

An informal picture. Fix a path factor F. Every other path factor G can be described by a height map φ on the cells, defined precisely below. φ is 0 outside the shape. Contiguous cells of equal height form regions, and the boundary between two regions is a *fence*. Walking along a fence, its edges alternately lie in F and not in F. G is $F \oplus X$, where X is the set of edges of all the fences of φ. φ changes by exactly ± 1 when crossing a fence. The sign is set by the arcs of F across the fence, which all point the same way, and φ rises by 1 when crossing in the direction of these arcs.

The height of φ at the hole is called the *class* of G. It can be interpreted as the number of fences that enclose, or loop around, the hole. Each such fence counts $+1$ if the arcs of F across it point toward the hole and -1 if they point away from it. Other fences, or loops, can be present, but since they do not encircle the hole they do not change the class. If there is no room to fit the class number of fences around the hole, the class is *empty* and can be rejected from consideration for a Hamiltonian path. Finding φ for a given class starts by assigning the class to the hole cells. The other cells then follow by a Bellman–Ford relaxation of the constraints *step* 0 *or* $+1$ *in the direction of the arc of* F, which raises every cell to the least value they allow (Fig. 14).

Put together, the algorithm runs Sheffield's loop once for every possible number k of fences around the hole, with k counted with its sign. Since $|k| \le |U|$, at most $2|U| + 1$ values need to be tried, and $(2|U| + 1)^2$ pairs with two holes. For each value the relaxation either finds a factor of that class, which is then the starting factor of the loop for that class, or shows that the class is empty. If



(a) class $h = 0$ (b) class $h = 1$ (c) $G_0 \oplus G_1$, $D/2$

Figure 13: Two classes at one leaf. V is the 4×5 block, $s = (1, 0)$ (deleted, the phantom q) and the inner endpoint $t = (2, 2)$ (deleted, grey hole cells). (a) The least factor G_0 of class $h = 0$, with 2 pieces. (b) The least factor G_1 of class $h = 1$, with 2 pieces. (c) $G_0 \oplus G_1$ (purple) is one alternating cycle around the hole. Each cell inside it shows $D(G_0, G_1)/2 = 1$, and the hole cells show 1 in red. The loop succeeds in class 0 and is stuck in class 1.

any factor of the class exists, the relaxation finds one, so no class that could hold a Hamiltonian path is skipped.

Definition 8.1 (reference factor, class)**.** Fix a phantom factor F for the hole t (the *reference factor*). For a factor G that agrees with F off the edges of V', the *class* of G is $h(G) = D(F, G)(t)/2 \in \mathbb{Z}$, read at the hole cell t (Lean `classSearchOne`, `class_mem_hRange`). With two holes t and r the class is the pair $(D(F, G)(t)/2,\ D(F, G)(r)/2)$.

All four hole cells carry the same value of $D(F, G)$, since no edge at t lies in $F \oplus G$. Since $F \oplus G$ is a union of alternating cycles and $D(F, G)$ changes by ± 2 exactly across its edges, $h(G)$ is the signed number of these cycles that go around the hole.

Classes in an example. Figure 13 shows a leaf of the 4×5 block with $s = (1, 0)$ and inner $t = (2, 2)$, in the branch $w = (3, 2)$, $x = (0, 0)$. Its factors fall into exactly two classes, $h = 0$ and $h = 1$. The least factors G_0 and G_1 of the two classes differ by a single alternating 12-cycle, which goes around the hole (Fig. 13c). Inside this cycle $D(G_0, G_1)/2 = 1$, and in particular it is 1 on the four hole cells. No sequence of flips turns G_0 into G_1, because a flip changes D only on the flipped cell and the hole cells are never flipped. The final test treats the two classes separately. It runs `classSearchOne` for $h = -18, \ldots, 18$, which returns nothing except for $h = 0$ and $h = 1$. For $h = 0$ the loop starts from G_0 with 2 pieces, flips one merging cell, and reaches one piece, so this leaf has a Hamiltonian path. Had $h = 0$ failed, the test would go on to $h = 1$. There the first phase from G_1 is stuck with 2 pieces, and the certificate shows that no factor of class 1 has one piece.

Lemma 8.2 (class implies `SolidFor`, `solidFor_of_holes`)**.** *If G and G' have the same class, then $D(G, G')(c) = 0$ for every square c that is not a cell of V'.*

The proof is in Appendix C.3. Lemma 8.2 supplies exactly the hypothesis `SolidFor` of Theorem 4.8.

Lemma 8.3 (the run keeps its class, `sf_flip`, `sf_flipCol`, `sf_reduceOutC2`, `HoleRun.lean`, `HoleTwo.lean`)**.** *A flip of a cell whose left and right sides have the same membership keeps $D(\cdot, X)$ off the flipped cell, for every X. Every flip made by the loop satisfies this condition when it is made. So every state of a run has the class of its start.*

The condition is checked flip by flip. A composite of degree-preserving flips can change the class, for example when two alternating cycles around the hole are toggled together.

The class search. Factors G that agree with F off the edges of V' correspond one to one to *admissible potentials* (Lean `Adm`, `HoleAdm.lean`). An admissible potential is an integer function φ on squares, zero outside the bounding box, whose jump across every edge e is 0, or equals the sign of the arc of F across e when e is an edge of V'. The factor is $G = F \oplus X_\varphi$ where X_φ is the set of edges of V' across which φ jumps, and $D(F, G) = 2\varphi$. Prescribing $\varphi(t) = h$ adds one more difference constraint. `classSearch` relaxes from below to the least admissible potential with the prescribed values, checks it, and returns $F \oplus X_\varphi$ (`HoleTry.lean`, `HoleRelax.lean`). `classSearchOne` V' F t h is `classSearch` with one prescribed value.

How the relaxation runs. The unknowns are the values $\varphi(c)$ on the squares c of the bounding box (`boxSquares`). Squares outside the box keep the value 0. For a side e between a lower or left square c^- and an upper or right square c^+, the rule *jump* 0 *or the sign* $\sigma(e)$ *of the arc of* F becomes the two inequalities

$$\min(0, \sigma(e)) \le \varphi(c^+) - \varphi(c^-) \le \max(0, \sigma(e)),$$

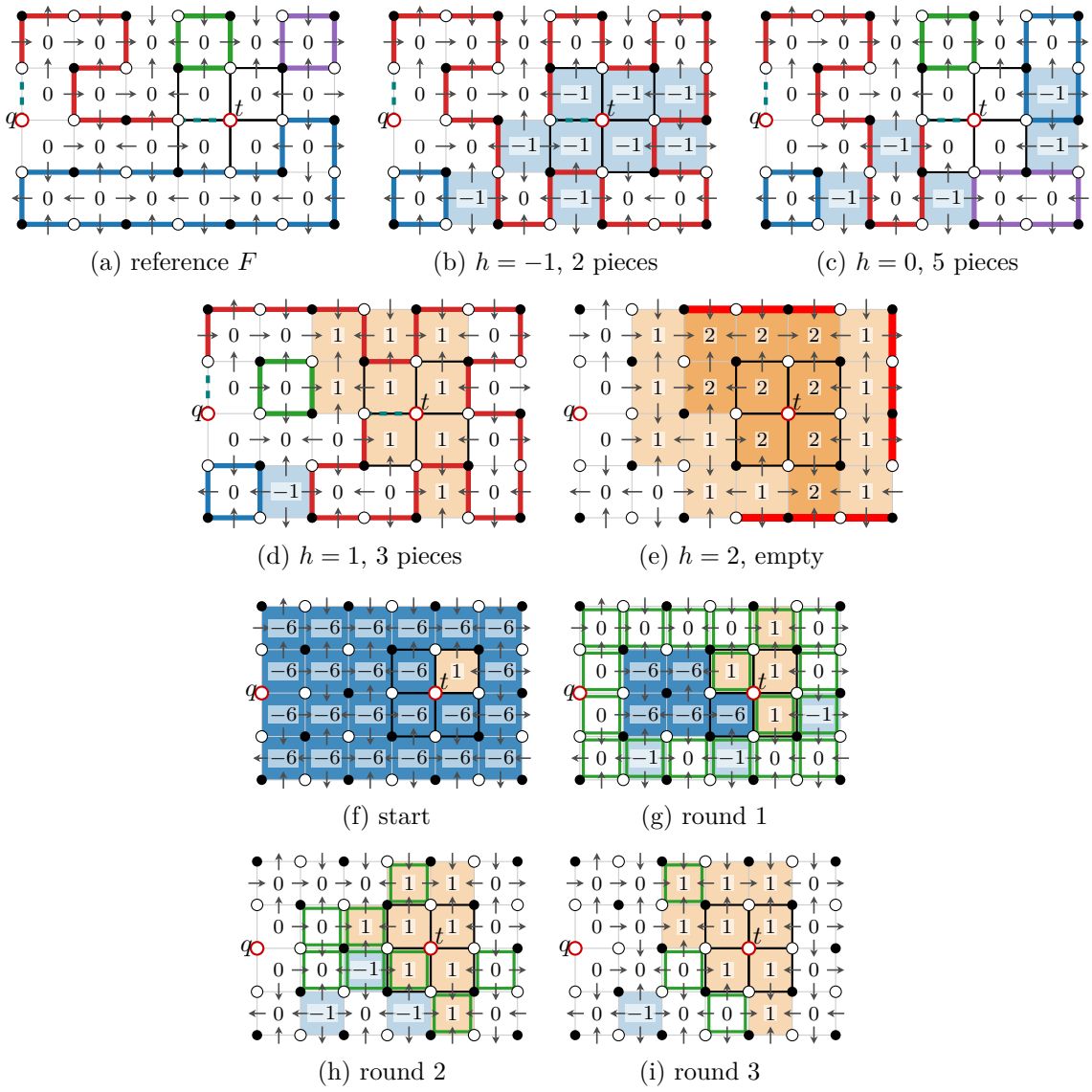
and a side that is not an edge of V' gets $0 \le \varphi(c^+) - \varphi(c^-) \le 0$. The prescribed hole cell starts at h and every other square at $-w$, where w is the width of the box (`kInitA`). No admissible potential is lower than $-w$, because it is 0 outside the box and changes by at most 1 from one square to the next. A round raises every square, all at once, to the largest value that its four sides force from the current values of its neighbours, or keeps it when that is lower (`lowerBound`, `roundA`). Values only rise, and the hole cell can rise above h as well. The rounds stop when nothing changes, at the latest after $|S| + 1$ rounds (`relax_bf`, below). If some admissible potential ψ with value h at the hole exists, every round stays at or below ψ, so the fixed point satisfies every inequality and still has the value h at the hole. `isAdmissible` checks exactly these two facts (`relax_fix`, `check_iff`). When the check fails the class is empty.

Figure 14 runs the search on a leaf of the 7×5 block. The factors of this leaf fall into three classes, $h = -1, 0, 1$. In each the least potential is a plateau of height h around the hole, and the boundary of the plateau is an alternating cycle of $F \oplus G$ around the hole. Other contours, such as the -1 cells of class 0, do not go around the hole and leave the class unchanged. For $h = 2$ the values forced around the hole reach the boundary of the box, where the step rule fails on nine sides, so the class is empty. In this leaf the loop reaches one piece in every nonempty class. In the leaf of Fig. 13 the loop succeeds in class 0 and is stuck in class 1.

Theorem 8.4 (`classSearchOne_sound`, `classSearchOne_complete`, `classSearchOne_solidFor`, `classSearchOneCost_le`)**.** *A factor returned by* `classSearchOne` *has class* h*. If some factor has class* h*,* `classSearchOne` *returns a factor* G *with* `SolidFor` V' G G' *for every* G' *of class* h*. If the bounding box of* V' *has width and height below* $|V'|$*, the cost is at most* $1000(|W| + 1)^4$*.*

Two facts give this bound. The potentials are stored in an array indexed by box position, so each lookup costs a constant (`relaxA`, `toFA`). The array relaxation simulates the list relaxation of the correctness proof (`toFA_relaxA`, `classPotential_eq`). The number of rounds is bounded as in Bellman–Ford (`relax_bf`, `HoleRelax.lean`). Let ψ be a solution and f the current potential. A cell that rises in round $k + 1$ has a neighbour that rose in round k, and $\psi - f$ does not decrease along such a chain of cells. A chain longer than the number of cells repeats a cell, where $\psi - f$ would have to drop. So $|S| + 1$ rounds reach the fixed point, where S is the list of squares of the box, and the fuel is `kFuel` $= |S| + 1$. In general `classSearch` with k prescribed values costs at most $500(k + 1)(|W| + 1)^4$ (`classSearchCost_le`).

The bounding box is narrow whenever a Hamiltonian path exists (`narrow_of_ncomp`), and this is tested first (`isNarrow`). Classes lie in $[-|V'|, |V'|]$, so the list `classRange` of $2|V'| + 1$ values covers them.



(a) reference F (b) $h = -1$, 2 pieces (c) $h = 0$, 5 pieces

(d) $h = 1$, 3 pieces (e) $h = 2$, empty

(f) start (g) round 1

(h) round 2 (i) round 3

Figure 14: The class search on one leaf of the 7×5 block, $s = (0, 2)$ (deleted, phantom q), inner $t = (4, 2)$ (hole cells outlined), branch $w = (3, 2)$, $x = (0, 3)$. Every cell shows $\varphi = D(F, G)/2$, orange above 0 and blue below. Points are black or white by parity, and the small arrows are the arcs of the reference factor F, which fix the allowed step of φ across each side (0 or $+1$ along an arrow, 0 or -1 against it). Sides without an arrow are not edges of $U = V \setminus \{s, t\}$, among them every side at s and t, including the two phantom edges, so φ cannot step across them, and the phantom edges are the same in F and in every factor of the leaf. (a) The reference factor F from `findFactor`, where $\varphi = 0$. (b)–(d) The least factor of each nonempty class $h = -1, 0, 1$, with its pieces. The edge between the plateau and its surroundings is an alternating cycle of $F \oplus G$ around the hole. (e) For $h = 2$ the numbers are not the heights of a factor. They are the least values that the value 2 at the hole forces, and every admissible φ with this hole value would be at least as large. For example, the arrow from the hole cell $(4, 2)$ into $(4, 3)$ forces $\varphi(4, 3) \geq 2$, while the outside square above $(4, 3)$ has $\varphi = 0$ and the arrow there allows at most 1. On the red sides the least values already exceed what the border allows, so no admissible φ exists and the class is empty. (f)–(i) The relaxation for $h = 1$. The prescribed hole cell starts at 1 and all other squares at -6. Green squares rose in that round. Round 3 is the fixed point, which equals (d).

9 One inner endpoint

Let t be inner and let s have a missing king neighbour. The two-phantom core needs the out-vertices w and s Far with phantoms that are not opposed, and s needs a phantom. Both come from deletions on the s side (`holeCase`, `holeCaseWalk`, `HoleWalk.lean`).

1. Delete t and branch over its neighbour $w \neq s$ (the last edge of the path). The phantom of w is t.
2. Delete s and branch over its successor $x \neq w$. The new endpoint is x, and the deleted s is its phantom q.
3. While the current endpoint is within king distance 1 of w, or its phantom q is opposed to the phantom t of w, delete it again and branch over its successor. The phantom q is always the point deleted last.
4. When the endpoint and w are Far and the phantoms are not opposed (`isFar`, `phantomsOpposed`), run the final test `oneHoleFinalTest`. It enumerates classes, calls `classSearchOne`, and runs the two-phantom loop `twoPhantomLoop`.

The measure `wmuG` is the number of points of the current set within king distance 1 of w (at most 8), plus 1 while q is not within king distance 2 of w. It is at most 9, and each deletion in step 3 lowers it. An opposed continuation needs q at king distance 3 from w, which happens only right after step 2. So step 3 runs at most 9 times, and the walk `holeCaseWalk` has fuel 10.

Why the phantom is the point deleted last. Write r for the original s, and let P be the path of deleted points from r to q. Theorem 7.4 needs degree 2 at every inner point except the vortex.

Lemma 9.1 (`ws_deg`)**.** *For every factor G of the current two-phantom setting, $G \cup P$ has degree 1 at t, degree 1 at r, and degree 2 at every other inner point of the original V.*

The phantom q gets one edge from G and one from P. If the phantom were another missing point, q would keep degree 1 in $G \cup P$, and an inner q would be a second vortex. The point r has a missing king neighbour, so it is not inner. So $G \cup P$ has the single vortex t on the solid original set. Its arcs agree with those of G on the cells of the current set, so climbs of G do not close (`ws_acyc`, `acyc_ext`, Theorem 7.6).

Definition 9.2 (walk invariant `WS`)**.** `WS` records the original solid set, the inner t, the current set U, the endpoint s, its phantom q, the edges w–t and s–q, the deleted path P with the degree condition of Lemma 9.1, and the fact that every missing point of U joins t or r through missing points or escapes to the left.

The last field gives `SolidFor` within a class by Lemma 8.2. `ws_init` establishes `WS` after steps 1 and 2, and `ws_step` keeps it under one deletion.

Theorem 9.3 (`oneHoleCase_main`, `HoleWalkCost.lean`)**.** *For solid V, inner t, and $s \neq t$ with a missing king neighbour, `oneHoleCase` decides the problem within $2\,513\,000\,(|V|+3)^5$ steps.*

10 Two inner endpoints

When s is inner too, the same walk applies with $r = s$ inner. Now $G \cup P$ has two vortices, t and r. Two things change.

Classes are pairs. The final test `twoHoleFinalTest` enumerates pairs (h_1, h_2) in `classRange`^{2}, at most $(2|U|+1)^2$ of them, and calls `classSearch` with two prescribed values (`kFind2_solidFor`, `HoleTwoV.lean`).

No closed climb with two vortices. A single potential of the shape in Theorem 7.6 fails. When one step can change both labels, their changes add up and can cancel the gain of τ. The fix is to give each vortex a label with three sectors, placed so that no step crosses boundaries of both.

Theorem 10.1 (height with two slits, `height2`, `TwoVortex.lean`)**.** *Let V be solid with two distinct inner points v_1, v_2 of degree 1 in H, and degree 2 at every other inner point. Let F_1, F_2 be unit flows from v_1, v_2 to a common far point. Then there are τ and $\delta_1, \delta_2 \in \{2, -2\}$ such that across every side e between cells, oriented from lower or left to upper or right, the jump of τ is the arc (± 1) minus $\delta_1 F_1(e) + \delta_2 F_2(e)$.*

The slits are chosen by the relative position of the vortices.

- *Different rows*, $v_1 = (a_1, b_1)$ below $v_2 = (a_2, b_2)$. Both slits go left along the rows of the vortices.
- *One row*, $v_1 = (a_1, b)$ left of $v_2 = (a_2, b)$. The slit of v_1 goes left along row b. The slit of v_2 goes down along column a_2 (Lean `SlitD`).

Definition 10.2 (three-sector labels)**.** For a square $c = (x, y)$ define (Lean `labD`, `labU`, `labL`, `labR`)

$$\mathrm{lab}_D^{(a,b)}(c) = \begin{cases} 3 & y \geq b \\ 1 & y < b,\ x < a \\ 2 & y < b,\ x \geq a \end{cases} \qquad \mathrm{lab}_U^{(a,b)}(c) = \begin{cases} 1 & y < b \\ 2 & y \geq b,\ x \geq a \\ 3 & y \geq b,\ x < a \end{cases}$$

$$\mathrm{lab}_L^{(a,b)}(c) = \begin{cases} 1 & x \geq a \\ 2 & x < a,\ y \geq b \\ 0 & x < a,\ y < b \end{cases} \qquad \mathrm{lab}_R^{(a,b)}(c) = \begin{cases} 2 & x < a \\ 0 & x \geq a,\ y < b \\ 1 & x \geq a,\ y \geq b \end{cases}$$

For different rows, $\ell_1 = \mathrm{lab}_D^{v_1}$ and $\ell_2 = \mathrm{lab}_U^{v_2}$. For one row, $\ell_1 = \mathrm{lab}_L^{v_1}$ and $\ell_2 = \mathrm{lab}_R^{v_2}$.

Each label has three boundary rays from its vortex. Across its slit it changes by ± 2, and across the other two rays by ± 1.

- lab_D: the row $y = b$ (the left part is the slit) and the downward ray $x = a$, $y < b$.
- lab_U: the row $y = b$ (the left part is the slit) and the upward ray $x = a$, $y > b$.
- lab_L: the column $x = a$ and the left ray $y = b$, $x < a$ (the slit).
- lab_R: the column $x = a$ (the lower part is the slit) and the right ray $y = b$, $x > a$.

Figure 15 shows both configurations with the computed labels.

Theorem 10.3 (`arc_vortex2`)**.** *Under the hypotheses of Theorem 10.1, H has no closed walk of arc steps between cells of V in a fixed direction.*

Proof. Fix the direction o, put $g = \pm 1$ as in Theorem 7.6, and take τ, δ_1, δ_2 from Theorem 10.1 with the slits above. For a step from cell c across side e to c', let $k_i \in \{-1, 0, 1\}$ record a crossing of slit i. For a left slit, $k_i = 1$ for an upward crossing and $k_i = -1$ for a downward one. For the downward slit, $k_2 = 1$ for a leftward crossing and $k_2 = -1$ for a rightward one (Lean `kL`, `kD`). Then

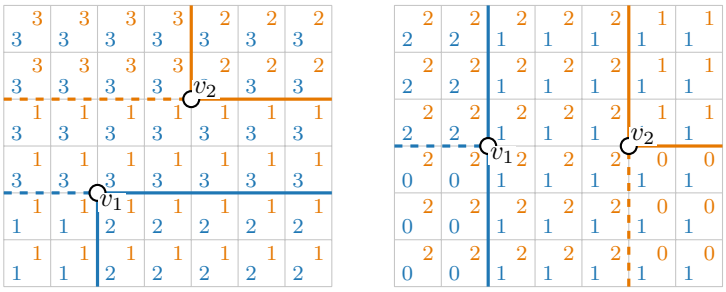


Figure 15: The three-sector labels of two vortices (open circles), computed from `labD`, `labU` (left) and `labL`, `labR` (right). Each cell shows ℓ_1 (blue, lower left) and ℓ_2 (orange, upper right). Solid lines are label boundaries with jump ± 1, and dashed lines are the slits, with jump ± 2. Left: different rows. Right: one row. No side lies on lines of both colours.

$\tau(c') - \tau(c) = (\pm 1) + \delta_1 k_1 + \delta_2 k_2$, where the first term is $+1$ exactly when the arc points from c to c' (`tau_step`). Put $\mu_i = -1$ if $g\delta_i = 2$ and $\mu_i = 1$ if $g\delta_i = -2$, so $g\delta_i = -2\mu_i$. Define

$$\Phi(c) = 3g\,\tau(c) + 2\mu_1 \ell_1(c) + 2\mu_2 \ell_2(c).$$

For a step whose arc points in direction o, $g\Delta\tau = 1 - 2\mu_1 k_1 - 2\mu_2 k_2$, so

$$\Delta\Phi = 3\,(1 - 2\mu_1 k_1 - 2\mu_2 k_2) + 2\mu_1 \Delta\ell_1 + 2\mu_2 \Delta\ell_2.$$

Two geometric facts hold for every unit step (`geo_rows`, `geo_same`).

(G1) If $k_i = 0$ then $|\Delta\ell_i| \le 1$. If $k_i \neq 0$ then $\Delta\ell_i = 2k_i$.

(G2) At most one of the pairs $(k_1, \Delta\ell_1)$ and $(k_2, \Delta\ell_2)$ is nonzero.

(G1) holds because the slit is exactly the boundary with jump 2, and the jump is $+2$ in the direction counted by $k_i = 1$. (G2) holds because the boundary rays of the two labels share no side. For different rows, the two rows $y = b_1$ and $y = b_2$ are different lines, the downward ray of v_1 lies below b_1, and the upward ray of v_2 lies above $b_2 > b_1$. For one row, the columns $x = a_1$ and $x = a_2$ differ, the left ray lies left of a_1, and the right ray lies right of $a_2 > a_1$. Now there are three cases.

- Both pairs are zero. Then $\Delta\Phi = 3$.
- Pair i is nonzero with $k_i = 0$. Then $\Delta\Phi = 3 + 2\mu_i \Delta\ell_i \ge 3 - 2 = 1$.
- Pair i is nonzero with $k_i = \pm 1$. Then $\Delta\ell_i = 2k_i$ and $\Delta\Phi = 3 - 6\mu_i k_i + 4\mu_i k_i = 3 - 2\mu_i k_i \ge 1$.

So Φ rises by at least 1 along every step, and no closed walk exists (`arc_of_pot`). The case of v_2 below v_1, or right of it, follows by exchanging the names (`arc_vortex2`). □

Theorem 10.4 (`twoHoleCase_main`, `HoleTwoVCost.lean`). *For solid V and distinct inner s, t, `twoHoleCase` decides the problem within $5\,217\,000\,(|V| + 3)^6$ steps.*

11 The algorithm and its running time

`decideHamPath` tests whether each endpoint is inner (`isInner`) and dispatches as in Table 3.

s	t	algorithm	Lean theorem	bound
not inner	not inner	`boundaryEndpoints`	`boundaryEndpoints_main`	$1\,060\,000\,(\|V\|+3)^4$
not inner	inner	`oneHoleCase`	`oneHoleCase_main`	$2\,513\,000\,(\|V\|+3)^5$
inner	not inner	`oneHoleCase`, s,t swapped	`oneHoleCase_main`	$2\,513\,000\,(\|V\|+3)^5$
inner	inner	`twoHoleCase`	`twoHoleCase_main`	$5\,217\,000\,(\|V\|+3)^6$
all cases, plus 10 for the dispatch			`decideHamPath_main`	$5\,220\,000\,(\|V\|+3)^6$

Table 3: The four cases of `decideHamPath` and their proved bounds. Swapping the endpoints in the third row uses that a path read backwards is a path (`hp_swap`).

Where the exponents come from. One round of the fast loop (a phase and its reduction) costs at most $3000(|V|+3)^3$ (`twoPhantomLoopCost_le`). The loop runs at most $|V|+2$ rounds, which gives degree 4 (`twoPhantomTestCost_le`). In the hole cases the loop runs once per class, and the class search costs $O(N^4)$ per class. One hole has $2|U|+1$ classes, which gives degree 5. Two holes have $(2|U|+1)^2$ class pairs, which gives degree 6. The leading constants count the final tests of the deletion walks. In the hole cases the two deletions before the walk give $4 \cdot 4 = 16$ walk starts, and each walk makes at most 13 final tests and visits at most 21 nodes (Section 11), so 208 final tests of cost at most $12\,000\,(|U|+3)^5$ (one hole) or $25\,000\,(|U|+3)^6$ (two holes). In case A the first step of the walk has no deleted neighbour yet, so a walk makes at most $4 \cdot 13 = 52$ final tests, and the possible deletion of s before it gives 4 walks.

Caveats. The cost functions charge unit cost for operations on finite sets of points and edges. A machine implementation pays more for these operations, so its exponent can be larger, but it stays polynomial. The Lean statement does not link `decideHamPathCost` to `decideHamPath` formally. The link is the parallel structure of the two definitions, which the reader checks. The companion Umans–Lenhart formalization in the same repository uses a step-counting monad instead (Section 12).

The walk constant: the ring potential. Fix the walk's centre w and its hole t (in case A: the centre s and its phantom q). The king neighbours of w other than t form a path of 7 points around w, the *ring*, numbered $0,\dots,6$ (`pidx`, `ptAt`). From ring point i a lattice step goes to w, to t, to ring point $i \pm 1$, or outward to king distance 2; a corner of the ring has 2 outward steps and an edge midpoint 1 (`outI`, `cat_counts`). The walk continues only from a ring point, or from distance 2 with an opposed phantom; an outward step from the ring stops, since the phantom (the ring point just left) is not opposed. Let the weight of a present ring point be its number of outward steps. The leaf potential L of a walk state is 0 when at most 2 points remain, 1 at a stop, 13 in an opposed state, and on ring point i the weight of i plus the weights of the unbroken runs of present ring points on both sides of i (`Lp`). The 7 weights sum to 11, so $L \le 13$. A step along the ring to $i+1$ deletes i and leaves the child with the run after i only, an outward step is a stop worth 1, paid by the weight of i; so the children's potentials sum to at most L (`ring_step`). An opposed state at king distance 2 has four lattice steps: onto its phantom (absent), one toward w (potential at most 11), and two sideways, which stay at distance 2 and stop; so its children sum to at most $11+1+1 = 13$ (`opp_step`). Hence a walk makes at most 13 final tests; the node potential M with weights 1+(outward steps) bounds its nodes by 21 (`holeCaseWalkCost_pot`, `boundaryEndpointsWalkCost_pot`). All facts about the ring are checked by `decide` on the 4 rotations and 28 relative positions.

Pseudocode. DecideHamPath (Algorithm 1) chooses one of three deletion walks by testing which endpoints are inner. Each walk deletes endpoints until the two out-vertices are Far with phantoms that are not opposed, and then runs a final test. The final test builds a path factor with FindFactor, in the hole cases moves it into each class with ClassSearch, and runs the two-phantom loop TwoPhantomLoop. Each round of that loop is a RunPhase followed by ReducePieces.

The blocks below use the following conventions. Each procedure has the name of its Lean definition, capitalised, and the caption of its block gives the Lean file. Helper predicates and functions are set in sans serif and are described after each block with their Lean names. $\mathsf{Nbrs}(t)$ (Lean `nbrsL`) is the list of pairs (y, e) with y a lattice neighbour of t and e the edge t–y, in the order right, left, up, down. The cells of V are visited in lexicographic order (`cellList`). The procedures return Booleans, and true means that a Hamiltonian path exists. Every loop over a list stops at its first success, as `List.any`, `List.find?` and `List.findSome?` do in Lean. A loop "for n rounds" runs at most n times, and n is the fuel of the Lean definition. The Python reference implementation returns the path as a witness instead of true.

Algorithm 1 DecideHamPath (Lean `decideHamPath`, `HPFinal.lean`)

function DecideHamPath(V, s, t)
  if $\mathsf{IsInner}(V, s)$ **then**
    if $\mathsf{IsInner}(V, t)$ **then return** TwoHoleCase(V, s, t) ▷ both endpoints inner
    else return OneHoleCase(V, t, s) ▷ only s inner, read the path backwards
  else if $\mathsf{IsInner}(V, t)$ **then return** OneHoleCase(V, s, t) ▷ only t inner
  else return BoundaryEndpoints(V, s, t) ▷ neither endpoint inner

In Algorithm 1, $\mathsf{IsInner}(V, p)$ (Lean `isInner`) holds when the four squares with corner p are cells of V.

Algorithm 2 BoundaryEndpoints (Lean `boundaryEndpoints`, `TwoPhWalk.lean`)

function BoundaryEndpoints(V, s, t) ▷ s and t each have a missing king neighbour
  if $|V| \le 2$ **then return** [s is a lattice neighbour of t]
  if some $(q, e_{sq}) \in \mathsf{Nbrs}(s)$ has $q \notin V$ (take the first) **then** ▷ q becomes the phantom of s
    return BoundaryEndpointsWalk$(s, q, e_{sq}, 12, V, t)$
  for all $(x, e) \in \mathsf{Nbrs}(s)$ with $x \in V$, $x \neq t$ **do** ▷ delete s (phantom of x)
    if BoundaryEndpointsWalk$(x, s, e, 12, V \setminus \{s\}, t)$ **then return** true
  return false

In Algorithms 2 and 3, e_{sq} is the phantom edge s–q and e_{tp} is the phantom edge t–p. The fuel 12 allows the at most 11 deletion levels of Section 6. $\mathsf{IsUsablePhantom}(V, s, q, t, p)$ (Lean `isUsablePhantom`) holds when $p \notin V$, $p \neq q$, and the phantom p of t is not opposed to the phantom q of s, so p is a usable phantom. $\mathsf{IsFar}(t, s)$ (Lean `isFar`) holds when $t = s$ or $d_\infty(t, s) \ge 2$. $\mathsf{PhantomsOpposed}(a, m_a, b, m_b)$ (Lean `phantomsOpposed`) holds when the phantoms m_a of a and m_b of b are opposed (Definition 5.4).

In Algorithm 4 the point w is the neighbour of t on the path, e_{wt} is the edge w–t, and the deleted t is the phantom of w. The argument *finalTest* is a function of (w, e_{wt}) that returns the final test of the walk. In TwoHoleCase the original s is passed to TwoHoleFinalTest as the second hole r.

Algorithm 3 BoundaryEndpointsWalk (Lean `boundaryEndpointsWalk`, `TwoPhWalk.lean`)

function BoundaryEndpointsWalk($s, q, e_{sq}, \mathit{fuel}, V, t$) ▷ s keeps its phantom q, t moves
  if $\mathit{fuel} = 0$ **then return** false
  if $|V| \le 2$ **then return** [s is a lattice neighbour of t]
  if some $(p, e_{tp}) \in \mathsf{Nbrs}(t)$ has $\mathsf{IsUsablePhantom}(V, s, q, t, p)$ (take the first) **then**
    if $\mathsf{IsFar}(t, s)$ **then** ▷ p is usable and t is Far from s
      return TwoPhantomTest($V, s, t, p, q, e_{tp}, e_{sq}$) ▷ the final test
  for all $(x, \cdot) \in \mathsf{Nbrs}(t)$ with $x \in V$, $x \ne s$ **do** ▷ delete t, x is the next endpoint
    if BoundaryEndpointsWalk($s, q, e_{sq}, \mathit{fuel} - 1, V \setminus \{t\}, x$) **then return** true
  return false

Algorithm 4 HoleCase (Lean `holeCase`, `HoleWalk.lean`), OneHoleCase (Lean `oneHoleCase`, `HoleWalk.lean`) and TwoHoleCase (Lean `twoHoleCase`, `HoleTwoV.lean`)

function HoleCase($\mathit{finalTest}, V, s, t$) ▷ t is inner
  if $|V| \le 2$ **then return** [s is a lattice neighbour of t]
  for all $(w, e_{wt}) \in \mathsf{Nbrs}(t)$ with $w \in V$, $w \ne s$ **do** ▷ delete t (phantom of w)
    if $|V \setminus \{t\}| \le 2$ **then**
      if s is a lattice neighbour of w **then return** true
    else
      for all $(x, e) \in \mathsf{Nbrs}(s)$ with $x \in V \setminus \{t\}$, $x \ne w$ **do** ▷ delete s (phantom of x)
        if HoleCaseWalk($\mathit{finalTest}(w, e_{wt}), w, t, 10, V \setminus \{t, s\}, x, s, e$) **then return** true
  return false

function OneHoleCase(V, s, t) ▷ t inner, s not inner
  $\mathit{finalTest}(w, e_{wt})(U, s', q, e_{sq}) \leftarrow$ OneHoleFinalTest($U, s', w, t, q, e_{wt}, e_{sq}$)
  return HoleCase($\mathit{finalTest}, V, s, t$)

function TwoHoleCase(V, s, t) ▷ the original s is the second hole
  $\mathit{finalTest}(w, e_{wt})(U, s', q, e_{sq}) \leftarrow$ TwoHoleFinalTest($U, s', w, t, s, q, e_{wt}, e_{sq}$)
  return HoleCase($\mathit{finalTest}, V, s, t$)

Algorithm 5 HoleCaseWalk (Lean `holeCaseWalk`, `HoleWalk.lean`)

function HoleCaseWalk($\mathit{finalTest}, w, p_w, \mathit{fuel}, U, s, q, e_{sq}$) ▷ w keeps its phantom p_w, s moves
  if $\mathit{fuel} = 0$ **then return** false
  if $|U| \le 2$ **then return** [s is a lattice neighbour of w]
  if $\mathsf{IsFar}(w, s)$ and not $\mathsf{PhantomsOpposed}(w, p_w, s, q)$ **then** ▷ ready for the final test
    return $\mathit{finalTest}(U, s, q, e_{sq})$
  for all $(x, e) \in \mathsf{Nbrs}(s)$ with $x \in U$, $x \ne w$ **do** ▷ delete s (phantom of x)
    if HoleCaseWalk($\mathit{finalTest}, w, p_w, \mathit{fuel} - 1, U \setminus \{s\}, x, s, e$) **then return** true
  return false

In Algorithm 5 the phantom p_w of w is the deleted inner point t. The phantom q of s is the point deleted last, and e_{sq} is the edge s–q. The fuel 10 allows the at most 9 deletions of Section 9.

Algorithm 6 OneHoleFinalTest (Lean `oneHoleFinalTest`, `HoleWalk.lean`) and TwoHoleFinalTest (Lean `twoHoleFinalTest`, `HoleTwoV.lean`)

1: **function** OneHoleFinalTest$(U, s, w, t, q, e_{wt}, e_{sq})$ ▷ hole at t
2: **if** not IsNarrow(U) **then return** false ▷ a wide set has no Hamiltonian path
3: $R \leftarrow$ FindFactor(U, d_{sw}) ▷ a path factor with ends s and w
4: **if** $R =$ none **then return** false
5: $F \leftarrow R \cup \{e_{wt}, e_{sq}\}$ ▷ the reference factor, phantoms t and q
6: **for** $h = -|U|, \ldots, |U|$ **do** ▷ every possible class (`classRange`)
7: $G \leftarrow$ ClassSearch$(U, F, [t], [h])$ ▷ a factor of class h (`classSearchOne`)
8: **if** $G \neq$ none and TwoPhantomLoop$(U, t, q, |U| + 2, G)$ **then return** true
9: **return** false

10: **function** TwoHoleFinalTest$(U, s, w, t, r, q, e_{wt}, e_{sq})$ ▷ holes at t and r
11: **if** not IsNarrow(U) **then return** false
12: $R \leftarrow$ FindFactor(U, d_{sw})
13: **if** $R =$ none **then return** false
14: $F \leftarrow R \cup \{e_{wt}, e_{sq}\}$
15: **for** $h_1 = -|U|, \ldots, |U|$ **do**
16: **for** $h_2 = -|U|, \ldots, |U|$ **do** ▷ every possible pair of classes
17: $G \leftarrow$ ClassSearch$(U, F, [t, r], [h_1, h_2])$
18: **if** $G \neq$ none and TwoPhantomLoop$(U, t, q, |U| + 2, G)$ **then return** true
19: **return** false

In Algorithm 6, IsNarrow(U) (Lean `isNarrow`) holds when the width and the height of the bounding box of U are both less than $|U|$. The degrees d_{sw} are 1 at s and w and 2 elsewhere. The factor F is a two-phantom factor on $U \cup \{t, q\}$ with phantoms t of w and q of s.

In Algorithm 7, BoxSquares(V) (Lean `boxSquares`) lists the squares of the bounding box of V, and BoxWidth(V) (Lean `boxWidth`) is the width of that box. Let $\sigma(e)$ be the sign of the arc of F across e (+1 from the lower or left square to the upper or right one) when e is an edge of V, and 0 otherwise. LowerBound(V, F, φ, c) (Lean `lowerBound`) is the largest lower bound on $\varphi(c)$ given by the four neighbours of c under the constraint that each jump lies between 0 and $\sigma(e)$. It is the maximum of $\varphi(c_{\downarrow}) + \min(0, \sigma)$ and $\varphi(c_{\leftarrow}) + \min(0, \sigma)$ across the bottom and left sides and of $\varphi(c_{\uparrow}) - \max(0, \sigma)$ and $\varphi(c_{\rightarrow}) - \max(0, \sigma)$ across the top and right sides. IsAdmissible (Lean `isAdmissible`) holds when every side of every square of the box has jump 0, or is an edge e of V with jump $\sigma(e)$, and $\varphi(a_i) = h_i$ for all i. JumpEdges(V, φ) (Lean `jumpEdges`) is the set of edges of V on sides of squares of the box across which φ jumps. The rounds bound $|box| + 1$ is `kFuel`. The potentials are stored in an array indexed by box position (`toFA`). With one hole, ClassSearch is the Lean `classSearchOne`.

In Algorithm 9, a bridge $b = (q_b, a_b, r_b)$ is a row of length r_b from the cell q_b in the frame with front a_b, and E_b is its last cell c_{r_b-1}. Its fixed pair is (c_{r_b-2}, c_{r_b-1}), and these two cells are *pair squares* (Lean `pairSqB`). ConnectedPairs(W, H) (Lean `connectedPairs`) is the set of pairs of points of W that lie in one piece of H. A merging cell is an alternating cut cell (`Alt`, `isCutCell`). FindRow (Lean `findRow`) scans the cells q of V in order and for each the sides a in the order bottom, top, left, right. It returns (q, a, r) for the first q that is critical in frame a with dark back and light

Algorithm 7 ClassSearch (Lean `classSearch`, `HoleTry.lean`, with `classPotential` and `relaxA`, `HoleRelax.lean`)

function ClassSearch($V, F, [a_1, \ldots, a_k], [h_1, \ldots, h_k]$) ▷ hole a_i gets class h_i
  $box \leftarrow \mathsf{BoxSquares}(V)$ ▷ squares of the bounding box, x-major
  for all $c \in box$ **do** ▷ start below every solution (`kInitA`)
    $\varphi(c) \leftarrow h_i$ for the first i with $a_i = c$, else $-\mathsf{BoxWidth}(V)$
  $\varphi(c) \leftarrow 0$ for every square $c \notin box$
  for $|box| + 1$ rounds **do** ▷ Bellman–Ford relaxation (`relaxA`)
    $\varphi'(c) \leftarrow \max(\varphi(c), \mathsf{LowerBound}(V, F, \varphi, c))$ for all $c \in box$
    if $\varphi' = \varphi$ on box **then break** ▷ fixed point, the least potential
    $\varphi \leftarrow \varphi'$
  if $\mathsf{IsAdmissible}(V, F, [a_i], [h_i], \varphi)$ **then return** $F \oplus \mathsf{JumpEdges}(V, \varphi)$ ▷ the factor of these classes
  else return none ▷ no factor has these classes

Algorithm 8 TwoPhantomTest (Lean `twoPhantomTest`, `LoopF.lean`) and TwoPhantomLoop (Lean `twoPhantomLoop`, `ReduceS.lean`)

function TwoPhantomTest($V, s, t, p, q, e_{tp}, e_{sq}$) ▷ phantoms p of t and q of s
  $R \leftarrow$ FindFactor(V, d_{st}) ▷ a path factor with ends s and t
  if $R =$ none **then return** false
  return TwoPhantomLoop($V, p, q, |V| + 2, R \cup \{e_{tp}, e_{sq}\}$) ▷ add the phantom edges

function TwoPhantomLoop($V, p, q, fuel, G$)
  $W \leftarrow V \cup \{p, q\}$
  for *fuel* rounds **do** ▷ each round joins two pieces
    if $\mathrm{ncomp}(W, G) = 1$ **then return** true ▷ one piece, a Hamiltonian path
    $(H, bridges, outcome) \leftarrow$ RunPhase($V, W, |W| + 1, G, [\,]$)
    if $outcome \in \{\mathsf{stuck}, \mathsf{fuel}\}$ **then return** false
    $G \leftarrow$ ReducePieces($V, W, H, bridges, outcome$) ▷ one piece fewer, no bridges
  return $[\mathrm{ncomp}(W, G) = 1]$

Algorithm 9 RunPhase (Lean `runPhase`, `PhaseS.lean`)

function RunPhase($V, W, fuel, H, bridges$) ▷ *bridges* is listed newest first
  for *fuel* steps **do**
    $conn \leftarrow \mathsf{ConnectedPairs}(W, H)$ ▷ all pairs of points in one piece
    $c \leftarrow$ the first merging cell of V that is not a pair square ▷ `findMergingCell`
    if $c \neq$ none **then return** $(H, bridges, \mathsf{merge}\ c)$ ▷ flipping c joins two pieces
    $b \leftarrow \mathsf{FindRow}(V, conn, H, bridges)$ ▷ a critical cell whose row is not stuck
    if $b =$ none **then return** $(H, bridges, \mathsf{stuck})$
    if r_b is odd **then return** $(H, bridges, \mathsf{odd}\ b)$ ▷ lowering the odd row joins two pieces
    if $\mathsf{FlipSplits}(W, H, b)$ **then** ▷ bridge step, undone or lowered later
      flip E_b in H, $bridges \leftarrow b :: bridges$ ▷ one piece more
    else return $(H, bridges, \mathsf{twist}\ b)$ ▷ even row, flipping E_b splits nothing
  return $(H, bridges, \mathsf{fuel})$

front (`isCritical`), is not a square of any bridge, and has a row of length r that is not stuck (`scanRow`). `scanRow` steps $j = 1, 2, \ldots$ up to $|V| + 1$ times. It ends with a row of length j when the front arc of c_{j-1} differs from that of q, and it ends stuck when c_j is not a cell of V or is a cut cell. $\mathsf{FlipSplits}(W, H, b)$ (Lean `flipSplits`) holds when, after the flip of E_b, the corners b_1 and f_1 of E_b lie in different pieces.

Algorithm 10 ReducePieces (Lean `reducePieces`, `ReduceS.lean`) and UndoBridges (Lean `applyAll` and `undoPlan`)

function ReducePieces$(V, W, H, [b_1, \ldots, b_k], \mathit{outcome})$ ▷ b_1 is the newest bridge
  $H_0 \leftarrow H$
  for $i = 1, \ldots, k$ **do** ▷ the states before each bridge (`bridgeStatesConn`)
    $H_i \leftarrow H_{i-1}$ with E_{b_i} flipped, $\mathit{conn}_i \leftarrow \mathsf{ConnectedPairs}(W, H_i)$
  if $\mathit{outcome} = \mathsf{merge}\ c$ **then**
    $X \leftarrow$ the first cell of V with $\mathsf{IsGoodMergingCell}(W, H, [b_i], [\mathit{conn}_i], X)$, else c ▷ Theorem R
    $(u, v) \leftarrow \mathsf{MergePair}(H, X)$, flip X in H ▷ half bridge
  else if $\mathit{outcome} = \mathsf{odd}\ b$ or $\mathit{outcome} = \mathsf{twist}\ b$ **then**
    $(u, v) \leftarrow \mathsf{RowStartPair}(b)$
    flip the cells $c_{r_b-1}, \ldots, c_1, c_0$ of the row of b in H ▷ half bridge, lower the row
  else return H
  return UndoBridges$(H, [b_1, \ldots, b_k], [\mathit{conn}_1, \ldots, \mathit{conn}_k], (u, v))$

function UndoBridges$(H, [b_1, \ldots, b_k], [\mathit{conn}_1, \ldots, \mathit{conn}_k], (u, v))$ ▷ (u, v) is the pending pair
  for $i = 1, \ldots, k$ **do** ▷ the plan (`undoPlan`)
    if u and v are connected in conn_i **then** ▷ joined before b_i was made
      $\mathit{op}_i \leftarrow$ lower b_i, $(u, v) \leftarrow \mathsf{RowStartPair}(b_i)$
    else $\mathit{op}_i \leftarrow$ undo b_i
  for $i = 1, \ldots, k$ **do** ▷ apply the plan (`applyAll`)
    if op_i lowers b_i **then** flip the cells $c_{r_{b_i}-2}, \ldots, c_0$ of b_i in H
    else flip E_{b_i} in H ▷ back to the state before b_i
  return H

In Algorithm 10, conn_i is the connection set of the state before bridge b_i was made. The half bridge joins the pieces of u and v, and UndoBridges removes the bridges while keeping that join. $\mathsf{MergePair}(H, X)$ (Lean `mergePair`) is the pair (b_1, f_1) of X in the frame with front top when the bottom of X is dark and front right otherwise (`altFrame`). $\mathsf{RowStartPair}(b)$ (Lean `rowStartPair`) is the pair (b_1, f_1) of q_b in the frame a_b. $\mathsf{IsGoodMergingCell}$ (Lean `isGoodMergingCell`) tests that X is alternating, cut, and not a pair square, and that every bridge b_i whose square beyond E_{b_i} is X is lowered by the plan started from $\mathsf{MergePair}(H, X)$, so X is good in the sense of Theorem R. The plan and the operations are computed and applied in the order $b_1, \ldots, b_k$, newest bridge first.

In Algorithm 11, a point (x, y) is black when $x + y$ is even and white otherwise. $\mathsf{AltNbrs}(V, F, u)$ (Lean `nbr`) is the set of points joined to u by an edge of V that is not in F when u is black and in F when u is white. So layer_k is the set of points reached from a deficient black point by an alternating walk of at most k steps. $\mathsf{IsBackEdge}(V, F, L, v, e)$ (Lean `Bwd`) holds when e is an edge of V at v whose other end is in L, with $e \in F$ when v is black and $e \notin F$ when v is white. It reads the factor F of the start of the round, not F'. The edges at v are tried in the order right, left, up, down. Each round toggles the edges of one augmenting walk from y back to layer_0, and the rounds stop when no deficient white point is reachable.

Algorithm 11 FindFactor (Lean `findFactor` with `augLoop`, `augStep` and `layerAcc`) and AugmentingWalk (Lean `walk`), `FindFactor.lean`

function FindFactor(V, d) ▷ a factor with degrees d, or none
  $F \leftarrow \emptyset$
  for $|V| + 1$ rounds **do** ▷ one augmentation per round (`augLoop`)
    $layer_0 \leftarrow \{z \in V : z \text{ black},\ \deg_F z < d(z)\}$ ▷ deficient black points
    for $k = 1, \ldots, |V|$ **do** ▷ `layerAcc`
      $layer_k \leftarrow layer_{k-1} \cup \mathsf{AltNbrs}(V, F, layer_{k-1})$
    $y \leftarrow$ the least white $y \in layer_{|V|}$ with $\deg_F y < d(y)$ ▷ a deficient white end
    if no such y **then break** ▷ no augmenting walk is left
    $F \leftarrow \text{AugmentingWalk}(V, F, [layer_{|V|-1}, \ldots, layer_0], y)$
  if $\deg_F z = d(z)$ for all $z \in V$ **then return** F
  else return none

function AugmentingWalk($V, F, [L_{n-1}, \ldots, L_0], v$) ▷ walk from v back to a black start
  $F' \leftarrow F$
  for $k = n - 1, \ldots, 0$ **do**
    if $v \notin L_k$ **then** ▷ step back into layer k
      $e \leftarrow$ the first edge e at v with $\mathsf{IsBackEdge}(V, F, L_k, v, e)$
      if $e =$ none **then return** F'
      $F' \leftarrow F' \setminus \{e\}$ if v is black, else $F' \cup \{e\}$ ▷ toggle e
      $v \leftarrow$ the other end of e
  return F'

12 Formal verification

Structure. The proof is in the Lean 4 library `SolidGridHam` [3] with Mathlib [18]. The toolchain is pinned to `leanprover/lean4:v4.35.0-rc2` (`lean/lean-toolchain`), and Mathlib to the revision in `lean/lake-manifest.json`. The s–t path development is in `lean/SolidGridHam/Path/`, 98 files and about 27,600 lines. The definitions in the statement are in `lean/SolidGridHam/Basic/`. It builds on the grid, height and strip libraries of the companion Umans–Lenhart formalization (`ulhc_main`) in the same repository. Appendix B maps the ideas of this paper to modules.

Axioms. The command `#print axioms` applied to `SolidGridHam.HP.decideHamPath_main` lists `propext`, `Classical.choice` and `Quot.sound`. The same holds for `stuck_minimal` and `arc_vortex2`. These outputs were obtained from the built library on 2026-10-17. No file of the library contains `sorry`, and the build has no warnings.

Trusted definitions. Lean checks every step of the proof. The reader must check that the statement means what it should. The definitions in the statement are `Pt` ($\mathbb{Z}\times\mathbb{Z}$), `Edge`, `IsEdge`, `deg`, `IsFactor` (`Grid.lean`), `dST` (`StartPF.lean`), `ncomp` (`Graph.lean`) and `Solid` (`Grid.lean`). The definition of `Solid` was also cross-checked by computation against "all interior faces have unit area". For the step bound the reader must also check that `decideHamPathCost` and the cost functions it calls follow the structure of `decideHamPath`, and accept the primitive costs (for example `flipCost` $= 8$ in `CostStrips.lean`, `connBCost` in `CostConn.lean`). The algorithm `decideHamPath` itself is computable.

Implementation. The repository contains reference implementations in Python, C and JavaScript, each with a command line front end. The Python implementation has two modes. Its reference mode ports each definition on the call path of `decideHamPath` as one function with the same name, the same control flow, the same fuel constants and the same iteration orders. Its fast mode adds answer-preserving speedups, and the C and JavaScript implementations port the fast mode. All of them return the path on a "yes" answer. The tests compare the modes and the implementations with each other and with an exact brute force. The implementations are not verified.

How to check. With `elan` installed, run `lake exe cache get` and `lake build` in `lean/`. Then run `lake env lean` on a file containing `import SolidGridHam.Path.HPFinal` and `#print axioms SolidGridHam.HP.decideHamPath_main`.

13 Worked example

13.1 One complete run

Let V be the 4×4 block $\{0,1,2,3\}^2$ without its top-left point $(0,3)$, so $|V| = 15$, and let $s = (0,2)$ and $t = (2,2)$ (Fig. 16a). V is solid. The point t is inner and s is not, so `decideHamPath` (DecideHamPath, Algorithm 1) calls `oneHoleCase` V s t (OneHoleCase, Algorithm 4). This subsection follows that call from start to finish, in the order the algorithm executes it. The run visits two leaves. The first ends with a proof that it holds no Hamiltonian path, and the second finds one. The run then stops, because every branching stops at its first success.

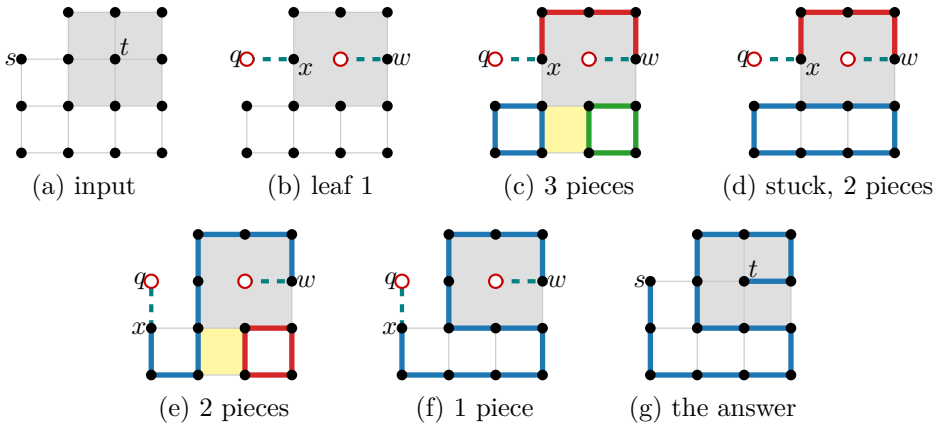


Figure 16: One run of `decideHamPath` on 15 points, in execution order. Red circles are deleted points, dashed teal edges are phantom edges, grey squares are the four cells around the inner endpoint t, and yellow cells are merging cells. Pieces are coloured blue, red and green. (a) The input. (b) Leaf 1. t is deleted and its first neighbour $w = (3, 2)$ gets the phantom t. s is deleted and its first neighbour $x = (1, 2)$ gets the phantom $q = s$. x and w are Far, so the walk stops at once. (c) The least factor of the only nonempty class, with 3 pieces, and its merging cell $(1, 0)$. (d) After the flip, 2 pieces, and the phase ends stuck. The certificate shows that no factor of the class has one piece, so leaf 1 answers no. (e) Leaf 2, the next successor $x = (0, 1)$ of s. The least factor of the class has 2 pieces and the merging cell $(1, 0)$. (f) After the flip, 1 piece. (g) Adding back s and t gives a Hamiltonian s–t path, and the run stops.

Deletions. The algorithm deletes t and tries its neighbours $w \neq s$ in the order right, left, up, down. The first is $w = (3, 2)$, which gets t as its phantom. It then deletes s and tries the neighbours $x \neq w$ of s in the same order. The first is $x = (1, 2)$, which gets $q = s$ as its phantom. The king distance of x and w is 2, so they are Far, and the phantoms are not opposed. The walk therefore stops before deleting anything else, and the final test `oneHoleFinalTest` (OneHoleFinalTest, Algorithm 6) runs on $U = V \setminus \{s, t\}$. This leaf asks whether V has a Hamiltonian path that starts with the edge s–$(1, 2)$ and ends with the edge $(3, 2)$–t.

Leaf 1, the class search. `findFactor` returns a factor with 2 pieces. The final test then tries the classes $h = -13, \ldots, 13$ in order. For each h, `classSearchOne` returns the least factor whose relative height at the hole cell of t is h, or nothing (Section 8). Only $h = 0$ gives a factor here. It has 3 pieces, a 4-cycle on $(0, 0), (1, 0), (0, 1), (1, 1)$, a 4-cycle on $(2, 0), (3, 0), (2, 1), (3, 1)$, and the path piece through the top two rows (Fig. 16c).

Leaf 1, the loop. The first phase finds the merging cell $(1, 0)$ between the two 4-cycles. The reduction flips it, which joins the two cycles into an 8-cycle. The second phase finds no merging cell and no critical cell that is not stuck, so it ends stuck with 2 pieces and no bridges. By the certificate (Theorem 4.8), every factor of this class has at least 2 pieces. Since $h = 0$ is the only nonempty class, leaf 1 has no Hamiltonian path, and the algorithm moves on.

Leaf 2. The next neighbour of s is $x = (0, 1)$, whose king distance to w is 3. Again the walk stops at once and only $h = 0$ gives a factor. This factor has 2 pieces, the 4-cycle on $(2, 0), (3, 0), (2, 1), (3, 1)$ and the path piece (Fig. 16e). The first phase finds the merging cell $(1, 0)$, and the flip leaves 1

piece. The loop answers yes, and adding back s and t gives the Hamiltonian path

$$(0,2),(0,1),(0,0),(1,0),(2,0),(3,0),(3,1),(2,1),(1,1),(1,2),(1,3),(2,3),(3,3),(3,2),(2,2).$$

The remaining branches are never evaluated.

13.2 Inside the leaves of a 4×4 instance

This subsection is not a run. It takes the 4×4 block $V = \{0,1,2,3\}^2$ with $s = (3,0)$ and inner $t = (1,1)$, and looks inside individual leaves to show a longer walk, the four factors of a class, and the slit height. Figures 7, 11 and 12 use this instance.

Consider the branch $w = (1,0)$ and $x = (2,0)$. Here $d_\infty(x,w) = 1$, so the walk deletes $(2,0)$. Its only successor other than w is $(2,1)$, again at king distance 1 from w, so the walk deletes $(2,1)$ too. Both successors of $(2,1)$, namely $(3,1)$ and $(2,2)$, are at king distance 2 from w, with phantom $(2,1)$, not opposed to t. The leaf for $(3,1)$ has $U = V \setminus \{t,(3,0),(2,0),(2,1)\}$, endpoint $(3,1)$, phantom $q = (2,1)$, and deleted path $P = (3,0)$–$(2,0)$–$(2,1)$ (Fig. 11, right). The set U has exactly one Hamiltonian path from $(3,1)$ to $(1,0)$,

$$(3,1),(3,2),(3,3),(2,3),(2,2),(1,2),(1,3),(0,3),(0,2),(0,1),(0,0),(1,0).$$

Adding back t and the deleted path gives a second Hamiltonian path of V from s to t.

At this leaf the point $(2,1)$ is inner in V and missing from U. In $G \cup P$ it has one edge from P and one phantom edge from G, so its degree is 2 (Lemma 9.1). Had the phantom of $(3,1)$ been the missing point $(3,0)$, the point $(2,1)$ would have degree 1 and would be a second vortex.

Factors and class. Take the leaf $w = (0,1)$, $x = (3,1)$. Here $d_\infty(x,w) = 3$, so the walk stops at once with $U = V \setminus \{t,s\}$ and phantom $q = s$. The setting is a two-phantom factor on $W = V$ with degree 1 at the phantoms t and q, containing w–t and x–q. The cells of U are $(0,2)$, $(1,2)$, $(2,1)$ and $(2,2)$. There are exactly four such factors, $G_0,\dots,G_3$.

factor	pieces	$D(G_0,G_k) \neq 0$ at	merging cells
G_0	2	none	$(1,2)$
G_1	3	$(2,1)$	$(1,2)$, $(2,1)$
G_2	1	$(1,2)$	none
G_3	2	$(2,1)$, $(1,2)$	$(2,1)$

Every nonzero value of $D(G_0,G_k)$ in the table equals 2. In particular $D(G_0,G_k) = 0$ at the hole cell t for all k, so all four factors lie in one class. Here `findFactor` returns G_1, so the reference factor is $F = G_1$. The admissible potentials are $D(G_1,G_k)/2$. They are 0 for G_1, -1 at $(2,1)$ for G_0, -1 at $(2,1)$ and 1 at $(1,2)$ for G_2, and 1 at $(1,2)$ for G_3, and 0 elsewhere. The least one with value 0 at t is that of G_0, so `classSearchOne` returns G_0 for $h = 0$ and nothing for $h \neq 0$.

The factor G_0. G_0 consists of the path piece

$$t,\ (0,1),\ (0,0),\ (1,0),\ (2,0),\ (2,1),\ (2,2),\ (2,3),\ (3,3),\ (3,2),\ (3,1),\ q$$

and the 4-cycle around the cell $(0,2)$ (Fig. 7, left). The cell $(1,2)$ has dark left and right sides and light top and bottom. Its left corners lie on the cycle and its right corners on the path, so it is a merging cell.

Arcs, slit height and potential. The deleted path is empty, so $G_0 \cup P = G_0$ on V. Its only inner point of degree 1 is t. The slit of $t = (1,1)$ is the side between the cells $(0,0)$ and $(0,1)$. Solving the jump equations of Theorem 7.4 on the nine cells is consistent only for $\delta = -2$. The resulting heights, shifted so the least is 0, are shown in Fig. 12 (right).

$y\backslash x$	0	1	2
2	1	2	1
1	0	1	2
0	3	2	1

Going once counterclockwise around t through the cells $(0,0), (1,0), (1,1), (0,1)$, the arcs give $-1, -1, -1, +1$, a total of -2. This is the vortex. For $o =$ `true` we have $g = 1$, $g\delta = -2$ and $\mu = 1$, so $\Phi = 2\tau + \mathrm{lab}$. Its values are $6, 5, 3$ on the bottom row (left to right), $3, 4, 6$ on the middle row and $5, 6, 4$ on the top row. Each of the twelve arcs between these cells goes from a smaller to a larger value of Φ. For example, the arc across the slit points from $(0,1)$ to $(0,0)$, and Φ goes from 3 to 6.

Flipping the merging cell $(1,2)$ of G_0 gives G_2, with one piece (Fig. 7).

14 The bulk case: endpoints far apart

Most of the paper is spent on moving the endpoints, because the proof must handle every position of s and t. When s and t are far apart, the walks stop at once and the work is elsewhere. This section describes what the algorithm does then, with one run on 34 points (Fig. 17).

The endpoints cost little. Deleting t (branch w) and s (branch x) moves each endpoint by one step. If s and t are at king distance at least 4, then w and x are at king distance at least 2, so they are Far. Their phantoms can be opposed only when w and x differ by exactly 2 along an axis (Definition 5.4), which needs king distance exactly 4 between s and t. So when s and t are at king distance at least 5, every walk stops at once and calls the final test. There are at most $4 \cdot 4 = 16$ such leaves, and they are tried one after another until one succeeds.

Inner endpoints become holes. An inner endpoint has all eight king neighbours in V. After its deletion it is a hole of U, surrounded by four cells (grey in Fig. 17). A factor of U with its two phantom edges has degree 1 at the hole, and its height field turns around it like a vortex (Section 7). Factors are grouped into classes by the relative height D at the cells around each hole. Within one class, the phase and the certificate work as in Sheffield's cycle algorithm, because the slit height and the quadrant label give a potential that rises along every climb (Section 7). Across classes they need not, so the final test runs Sheffield's loop once per class.

What the final test does. For one hole there are at most $2|U| + 1$ classes, and for two holes at most $(2|U| + 1)^2$ pairs. For each class, `classSearch` returns the least factor of the class, or nothing when the class is empty (Section 8). In the runs of Section 13.1 and of this section exactly one class, or class pair, is nonempty. The least factor of a class is the starting state of the loop. It can have more pieces than the factor `findFactor` found, because it is chosen for its heights and not for its pieces.

The moves of the loop. Each round of the loop is one phase and one reduction, and it lowers the number of pieces by one (Sections 4.3 and 4.7). A phase makes moves of three kinds.

- A *merge* flips an alternating cell whose corners lie in two pieces. The two pieces become one.
- A *bridge step* is used when no such cell exists. It takes a critical cell, follows its row to the last cell where the heights turn, and flips that last cell. The flip splits a piece, so the count goes up by one, but it exposes new cells.
- An *odd row* or a *twist* ends the phase like a merge, with a half bridge in place of the merging cell.

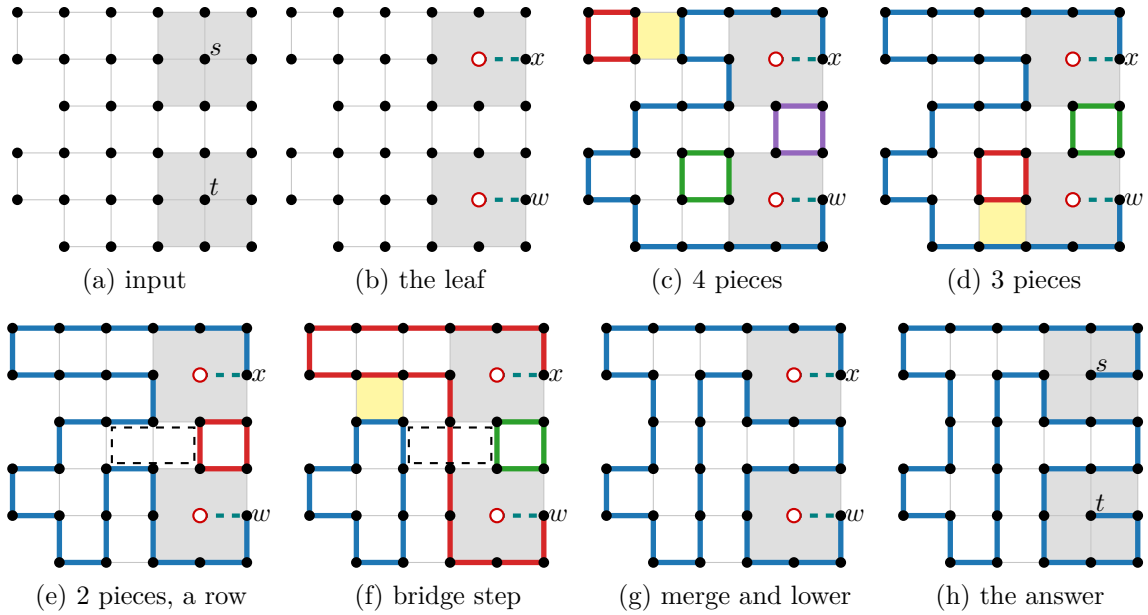
The reduction then flips the merging cell (or the half bridge), and walks back through the bridges, newest first. Each bridge is either lowered (the cells of its row before the last are flipped) or undone (its last cell is flipped back), as the plan of Section 4.3 decides. The net effect of a round is one piece fewer. If a phase ends stuck, no merge, bridge step or row is available, and the certificate shows that no factor of the class has fewer pieces. The loop then gives up on this class.

The example. Figure 17 shows a run on a 6×6 block with four points removed from the left column. Both endpoints $s = (4, 4)$ and $t = (4, 1)$ are inner, at king distance 3, so `decideHamPath` calls `twoHoleCase` (TwoHoleCase). The first branch, $w = (5, 1)$ and $x = (5, 4)$, already succeeds.

- Deleting t and s leaves U with 32 points and two holes. The walk takes no step, since w and x are at king distance 3.
- `findFactor` returns a factor with 2 pieces. The only nonempty class pair is $(0, 0)$, and its least factor has 4 pieces, a long path and three 4-cycles (Fig. 17c).
- Phase 1 flips the merging cell $(1, 4)$ and joins the top-left cycle to the path. Phase 2 flips $(2, 0)$ and joins the cycle at $(2, 1)$.
- Phase 3 starts with 2 pieces, the path and the cycle on $(4, 2), (5, 2), (4, 3), (5, 3)$ (Fig. 17e). No cell between them is alternating, so no merge is possible. The cell $(3, 2)$ is critical in the frame whose front is its left side. Its back, the right side, is dark and lies on the cycle. Its row is $(3, 2), (2, 2)$, of even length 2.
- The bridge step flips $(2, 2)$, the last cell of the row. This splits the path into two pieces, so there are now 3 (Fig. 17f). The split exposes the merging cell $(1, 3)$, and the phase ends with outcome merge.
- The reduction flips $(1, 3)$, which joins the two parts of the path again. The plan then lowers the bridge by flipping $(3, 2)$, which joins the cycle. One piece remains (Fig. 17g), and the loop answers yes.

In this round three cells were flipped, $(2, 2)$, $(1, 3)$ and $(3, 2)$. No single flip joins the last cycle to the path, and the bridge is what makes the two-step join possible. Adding back s and t gives the Hamiltonian path of Fig. 17h.

Where the running time goes. In the bound of `decideHamPath_main`, the deletions contribute only the constant 208. The polynomial comes from the final test. The class enumeration gives the factor $2|U| + 1$ per hole, the class search costs $O(N^4)$ per class, and the loop costs $O(N^3)$ per round for at most $|V| + 2$ rounds. With two holes this is $O(N^2) \cdot (O(N^4) + O(N^4)) = O(N^6)$.



(a) input (b) the leaf (c) 4 pieces (d) 3 pieces

(e) 2 pieces, a row (f) bridge step (g) merge and lower (h) the answer

Figure 17: One run of `decideHamPath` with both endpoints inner and far apart, in execution order. Grey squares are the cells around the two holes, red circles the deleted endpoints, dashed teal edges the phantom edges, yellow cells merging cells, and the dashed box a bridge row. (a) The input. (b) The leaf after deleting t (branch w) and s (branch x). (c) The least factor of the class pair $(0, 0)$, with 4 pieces, and the merging cell of phase 1. (d) After phase 1, 3 pieces, and the merging cell of phase 2. (e) After phase 2, 2 pieces. No merging cell exists. The critical cell $(3, 2)$ has the row $(3, 2), (2, 2)$ (dashed box). (f) The bridge step flips $(2, 2)$ and splits the path, and the merging cell $(1, 3)$ appears. (g) The reduction flips $(1, 3)$ and lowers the bridge by flipping $(3, 2)$, leaving 1 piece. (h) The Hamiltonian s–t path.

A Prior work and novelty

Complexity of grid Hamiltonicity. Itai, Papadimitriou and Szwarcfiter proved that Hamiltonian path and cycle are NP-complete in grid graphs, gave a linear-time algorithm for s–t paths in rectangular grid graphs, and posed the solid case [7]. Umans and Lenhart gave a polynomial algorithm for Hamiltonian cycles in solid grid graphs [20]. Arkin et al. [1] classified the complexity of Hamiltonian cycle over several grid families, and Demaine and Rudoy settled two of their open cases [4]. These works treat cycles.

Paths in solid grid graphs. Keshavarz-Kohjerdi [9] and Keshavarz-Kohjerdi and Bagheri [11] state that it remains open whether the Hamiltonian path problem is polynomially solvable in solid grid graphs. Their papers and related ones [10, 12] give conditions and linear-time algorithms for special shapes (L-, C-, F- and E-shaped grid graphs). A 2024 paper gives an algorithm for s–t paths in solid *supergrid* graphs, which also have diagonal edges [13]. A 2025 paper on rectangles with an L-shaped hole says that few results exist for grid graphs with holes [14]. The longest cycle problem in solid grid graphs is also described as open [16]. The literature search behind this paper found no claim of a polynomial algorithm for Hamiltonian paths in solid grid graphs between 2024 and 2026.

Sheffield's claim. Sheffield's paper [17] gives a height-function algorithm for Hamiltonian cycles in grid graphs with a bounded number of holes. Its introduction says "Similar results hold for

Hamiltonian paths with fixed endpoints and for other gridlike families of graphs embedded in n-holed tori." No proof, construction or section on paths follows. Throughout his §4 the degree function is identically 2. The papers listing the path problem as open do not cite this sentence. So the problem is better described as asserted without proof than as untouched. Two points show that the assertion needed work.

1. The local rule of Sheffield's §4.1 excludes an increasing sequence of faces around a vertex, using $\varphi(v) = 2$. At an endpoint $\varphi(v) = 1$ and the rule fails. For an inner endpoint this failure is the vortex of Section 7. Without a substitute, the climb in the final count need not terminate.
2. The written proof of the stuck case (§4.8) has a gap that is independent of endpoints (Section 4.5). The count step is false as written in some stuck states, though the theorem holds.

The present work gives the first written proof of the path case that we know of, and a machine check of it.

What is taken from Sheffield and what is new. The phase (rows, bridges, half bridges, the undo order), Lemmas 1 to 4, the stuck-state theorem and the hole classes are Sheffield's. His Theorem 8 indexes flip components of 2-factors with holes by the relative height on the holes, which is the idea behind Section 8. New here are the following.

- The column repair of §4.8 and the locally checkable certificate `Cert`.
- A computable choice rule for the merging cell (Theorem R) that avoids one unexcluded configuration of Lemma 4.
- Phantom endpoints, the two-phantom core under the Far condition, and the deletion walks.
- Classes relative to phantom and deleted endpoints, and the class search by relaxation, ported from the author's Lean formalization of Umans–Lenhart with holes.
- The handling of degree-one defects by slits and sector labels.

Vortices. That an endpoint of a path is a topological defect of a height field is standard in physics. In the Coulomb-gas description of compact polymers, cutting a loop into two strings creates a vortex and an antivortex of the height [8, §7]. In dimer models monomers act as magnetic charges that make the height additively multivalued [6, 5, 2]. These works use the defect for scaling and correlation computations. For domino tilings of regions with holes, Saldanha, Tomei, Casarin and Romualdo describe flip components through height sections on a covering [15]. Thurston's height functions for tilings are background [19]. We did not find the slit plus sector-label potential used to rule out closed climbs in the literature. The literature search does not prove that it is absent.

Formal verification. The literature search did not cover machine-checked graph algorithms. The present development builds on the author's own Lean formalization of the Umans–Lenhart cycle algorithm (`ulhc_main`, same repository).

B Lean module map

The files below are under `lean/SolidGridHam/Path/`. The definitions in the statement are in `lean/SolidGridHam/Basic/`, the Umans–Lenhart library that the path development

builds on is in `lean/SolidGridHam/Cycle/`, and lemmas on path factors and notches are in `lean/SolidGridHam/Path/Prelim/`.

idea	files	main results
cut-cell shapes, Lemma 2	`Shapes`, `E1`, `Agree`, `Rank`	Lemma 1 shapes, `ncomp_le_of_joins`
phase, bridges	`Bridges`, `Freeze`, `SPhase`, `SPhaseAlg`, `PhaseCount`	`regionOK_step`, `reach_count`
stuck ⇒ certificate	`Cert`, `CertOf`, `ColProof`, `SideProof`, `TriProof`, `StuckL4`	`cert_of_stuck`, `l4_of`
repaired §4.8	`Column`, `Move`, `MoveInv`, `ColAfter`, `SideAfter`, `E1After`, `Final`, `FinalLocal`, `Minimal`	`lowerCol_count`, `stuck_minimal`, `cert_count_gen`
undoing bridges	`Undo`, `UndoCount`, `Node`, `NoMid`, `HalfBridge`, `S4Rows`, `S4RowSpec`, `S4Merge`, `S4MergeII`, `S4TheoremR`, `Compat0`–`Compat3`, `CaseA1`, `CaseA2`, `CaseB`, `Overlap`, `Assemble`	`undo_count`, `goodCell_of`, `theoremR`
computable phase, cost	`SheffComp`, `SheffCost`, `SheffBoundary`	`planB_eq`, `goodB_iff`, `phI_step`
starting factor	`FindFactor`, `FindFactorCost`	`findFactor_some`, `findFactor_none`, `findFactorCost_le`
fast loop	`ConnSet`, `PhaseS`, `ReduceS`, `PhaseSCost`, `LoopF`	`isConnected_eq`, `runPhase_eq`, `reducePieces_eq`, `twoPhantomLoop_eq`, `twoPhantomLoopCost_le`, `twoPhantomTestCost_le`
two phantoms, deletion walk	`TwoPh`, `TwoPhS4`, `TwoPhCost`, `TwoPhWalk`	`hpSheffC2_main`, `del_iff`, `boundaryEndpoints_main`
heights, vortex, slit	`Height`, `SlitHeight`, `Final`	`height_exists`, `slit_height`, `arc_vortex`
classes	`HoleRun`, `HoleAdm`, `HoleRelax`, `HoleTry`, `HoleTryCost`	`solidFor_of_holes`, `sf_reduceOutC2`, `classSearchOne_sound`, `relax_bf`, `classPotential_eq`, `classSearchCost_le`
hole loop	`HoleTwo`, `HoleOne`	`runSC2_complete_hole`
one inner endpoint	`HoleWalk`, `HoleWalkCost`	`WS`, `ws_step`, `holeCaseWalk_correct`, `oneHoleCase_main`
walk constant	`WalkGeo`, `WalkCount`, `TwoPhWalkCost`	`ring_step`, `opp_step`, `holeCaseWalkCost_pot`, `boundaryEndpointsWalkCost_pot`
two vortices	`TwoVortex`, `HoleTwoV`, `HoleTwoVCost`	`height2`, `arc_vortex2`, `twoHoleCase_main`
dispatcher	`HPFinal`	`decideHamPath_main`

C Proofs deferred from the main text

C.1 Deletion (Lemma 6.1)

Let H be a Hamiltonian s–t path of V. The point t has degree 1, so exactly one edge w–t of H meets t. Since $|V| \geq 3$, $w \neq s$. Removing this edge gives a Hamiltonian s–w path of $V \setminus \{t\}$. Conversely, adding the edge w–t to a Hamiltonian s–w path of $V \setminus \{t\}$ gives a Hamiltonian s–t path of V.

C.2 The slit height (Theorem 7.4)

Let $w(e) = \pm 1$ be the arc weight on sides between two cells and 0 on other edges (Lean `wC`). The *defect* of a weight at a point p is the sum over the four edges at p with the signs of the balance relation (Lean `defect`). A potential with jumps w exists when the defect vanishes at every point (Lean `Potential.lean`). At inner points of degree 2 the defect of w is 0. At the vortex it is $\delta = \pm 2$. At points that are not inner, some edge at the point is not between two cells, and the defect can be nonzero. Each such point q is joined to a fixed point O far left by a path of edges not between two cells, because V is solid (`reach_far`). Subtracting the defect of q times a unit flow along this path removes the defect at q without changing any weight between two cells. The vortex's defect is removed by a unit flow along the slit, which runs straight left to O. The total defect is 0 (`sum_defect`), so after these corrections the weight is balanced everywhere. Its potential is τ. Across sides between two cells, τ jumps by the arc weight, plus δ on slit sides.

C.3 Class implies `SolidFor` (Lemma 8.2)

Let G, G' have the same class. Their phantom edges coincide, so $G \oplus G'$ consists of edges of V'. $D(G, G')$ is additive, with $D(G, G') = D(F, G') - D(F, G)$ (`D_sub`). So $D(G, G')(t) = 2(h(G') - h(G)) = 0$ at the hole cell. Let c be a square that is not a cell of V'. It has a missing corner m, and $D(G, G')(c) = D(G, G')(m')$ for every square m' at m, since no edge of $G \oplus G'$ meets m (`D_aroundW`). Two king-adjacent missing points share a square, so D is constant along a king path of missing points (`D_chainW`). If m joins t this way, $D(G, G')(c)$ equals the value at the hole, which is 0. Otherwise m escapes to a point left of V', where D is 0 because $G \oplus G'$ lies in V' (`pot_eq_zero_of_left`). With two holes the same argument uses both prescribed values.

C.4 Strand pairing (Lemma 4.12)

Index a column of length L from its bottom cell (index 0) to its top cell (index $L-1$). Let $v_0, \ldots, v_L$ and $w_0, \ldots, w_L$ be its left and right corners. Along the column the lateral $v_i v_{i+1}$ is dark exactly for odd i, the cross edge $v_i w_i$ $(0 < i < L)$ is dark exactly for even i, and by degree 2 the outward edge at v_i is dark exactly for odd i. Before lowering, for each odd i the left strand at height i is joined to the right strand at height i by the U-turn $v_i, v_{i+1}, w_{i+1}, w_i$. Lowering toggles every lateral and the two end sides, and toggles each inner cross edge twice. Afterwards the lateral $v_i v_{i+1}$ is dark exactly for even i. For each odd i the left strand at height i is joined to the right strand at height i by $v_i, v_{i-1}, w_{i-1}, w_i$. So the pairing of the outer strands is unchanged except at the two ends. In Lean this is `lowerCol_count`, with the strip count lemmas `strip_odd_pf` and `strip_even_rel_pf`.

C.5 The step relation of a bridge

Before the bridge step, E has dark laterals, light back and front, and is not cut (`rowStep_pre`, `RowStep.uncut`). After the flip, back and front of E are dark. The connectivity of the old state is the new connectivity with the classes of one back corner and one front corner of E merged, and

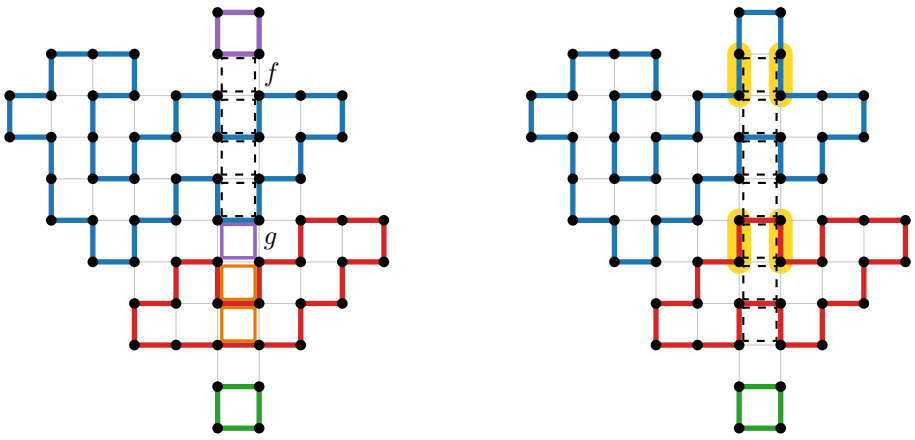
these two classes differ in the new state (`RowStep.split`). So each bridge step adds exactly one piece (Lemma I.1, `step_count`).

D Sheffield's stuck case and columns

Sheffield's argument. Sheffield proves that a stuck state H with $m + r$ pieces and r fixed pairs leaves every factor H' with the same degrees at least m pieces (his §4.8). He moves fixed pairs along rows toward H', until every cut component on which $D(H, H')$ is not constant contains a fixed-pair square. He then bounds the pieces of H' from below by the pieces of H minus the number of H'-edges that join two pieces of H, counted as the rank of a graph on the pieces (his Lemma 2). Two steps of this argument need columns.

A row stuck against a cut cell. The text treats a critical cell whose row runs into the outside of V. A row can also run into another cut cell g, as at $f = (2, 3)$ in Fig. 18a. Its row $f, (2, 2), (2, 1), (2, 0)$ (dashed) runs down into the cut cell $g = (2, -1)$ (purple), which is critical for a different cut, and the row of g continues down to the only fixed pair (orange). Moves of rows never change cut cells, so they cannot remove this situation. By the column lemma of Section 4.5 such chains are always straight, and the whole run from f to the fixed pair is one column of length 7.

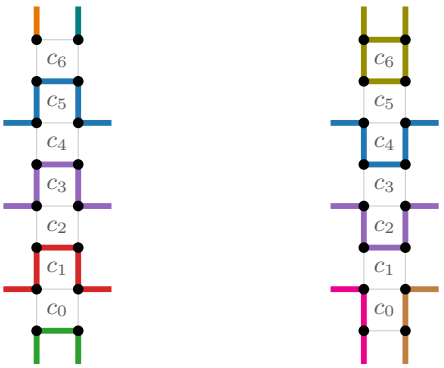
The count sees rerouting as joining. Lower the whole column of Fig. 18a. The result H' (Fig. 18b) has 3 pieces, one fewer than H, as it must with one fixed pair. Four edges of H' join two pieces of H (highlighted in yellow), at the two ends of f and of g, and they join three pieces of H into one, so the rank of Sheffield's graph is 2. The bound then gives only $4 - 2 = 2$ pieces for H', below the $m = 3$ that the theorem asserts. The edges at g do not join pieces of H'. They reroute strands through the column, and the only net join is at its bottom end. The same happens in every stuck state with such a column that was examined, nine in all. The theorem holds in each of them.



(a) the stuck state H, 4 pieces, 1 fixed pair (b) H', the column lowered, 3 pieces

Figure 18: Where Sheffield's count needs columns. (a) A stuck 2-factor H on 64 points with 4 pieces (colours) and one fixed pair (orange). The row of the critical cell f (dashed) runs into the cut cell g (purple), whose row continues to the fixed pair. (b) The competitor H', which is H with the column from f to the fixed pair (dashed) lowered. It has 3 pieces. The highlighted edges of H' join two pieces of H, and Sheffield's rank count treats them as two joins, while H' has only one piece fewer than H.

The fix. Columns behave like rows (Lemma 4.12). Along a column the laterals are dark exactly at odd heights, the inner cross sides at even heights, and strands leave the column sideways at odd heights. Lowering the column toggles the laterals and the two end sides, and the inner cross sides twice. Every inner strand is joined to the same partner before and after (Fig. 19), and only the strands at the two ends change. So lowering a column changes the pieces exactly as lowering one row of the same length. The proof replaces Sheffield's move by a move along a column (Section 4.5). After the moves every cut component on which $D \neq 0$ contains a fixed-pair square of the right sign, and the final count uses only these joins (`ncomp_le_of_joins`, `Rank.lean`). The result is `stuck_minimal`, and through `cert_inv` it gives Theorem 4.8.



(a) before lowering (b) after lowering

Figure 19: Strand pairing in a column of length 7, cells $c_0, \ldots, c_6$. Each colour is one strand of the factor inside the column, together with the outside strands it joins. (a) Before lowering, the strand leaving the column on the left at height 3 returns on the right at height 3, and likewise at height 5. (b) After lowering, the same pairs are joined through the other side of the U-turn. Only the strands at the two ends of the column change, and they get new colours.

E The starting factor as a maximum flow

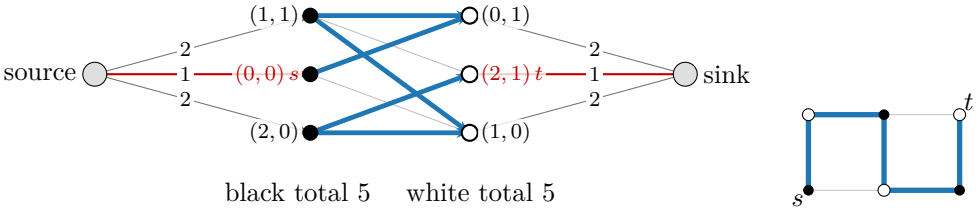
The network. Colour a point (x, y) *black* when $x + y$ is even and *white* otherwise. Every edge joins a black point to a white point, so the grid graph is bipartite. A factor with degrees $d \leq 2$ is then an integral flow in the following network. A source has an arc of capacity $d(z)$ to every black point z. Every edge of V is an arc of capacity 1 from its black end to its white end. Every white point z has an arc of capacity $d(z)$ to a sink. An integral flow that saturates every arc at the source and at the sink is a factor, and the edges that carry flow are its edges. Such a flow exists only if $\sum_{z \text{ black}} d(z) = \sum_{z \text{ white}} d(z)$.

What degree one changes. For Hamiltonian cycles $d = 2$ everywhere, and the condition is that V has as many black as white points. For a path factor d_{st} lowers the capacity to 1 at s and t, which removes one unit from the side of each endpoint. If $|V|$ is even the two sums agree only when s and t have different colours. If $|V|$ is odd they agree only when both have the colour that has one point more. In a leaf of the hole cases the network is built on U with capacity 1 at the two new endpoints x and w, and the phantom edges x–s and w–t are added afterwards.

The algorithm. `findFactor` is the Ford–Fulkerson method on this network without building it. A black point below its degree has spare capacity from the source, and a white point below its

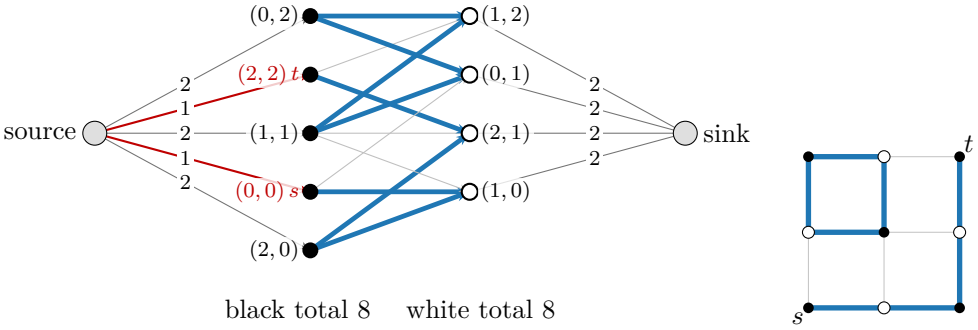
degree has spare capacity to the sink. `findFactor` builds the alternating layers from the deficient black points, stepping from a black point along an edge not yet chosen and from a white point along a chosen edge. It takes the least deficient white point that is reached and toggles the edges of the alternating path back to a deficient black point, which raises the degree of both ends by one. It does this at most $|V|+1$ times, starting from the empty edge set, with all choices fixed (`findFactor_some`). When no augmenting path is left the flow is maximum, and if some degree is still short no factor exists (`findFactor_none`). The cost is $O((|V|+1)^3)$ (`DFC.findFactorCost_le`).

Examples. Figure 20 draws the network for two small instances. In (a) V is the 3×2 block with 3 black and 3 white points, $s = (0,0)$ is black and $t = (2,1)$ is white. Both sides of the network have capacity $2+2+1 = 5$. Had s and t both been black, the black side would have capacity 4 and the white side 6, and no factor could exist. In (b) V is the 3×3 block with 5 black and 4 white points, and $s = (0,0)$ and $t = (2,2)$ are both black. The black side has capacity $2+2+2+1+1 = 8$ and the white side $4 \cdot 2 = 8$. The extra black point is balanced because both endpoints of capacity 1 are black. The flow drawn in (b) saturates every source and sink arc, so it is a factor, but it is an L-shaped path from s to t together with a 4-cycle in the upper left. A maximum flow only controls degrees, not connectivity, and the starting factor usually has several pieces. Joining them is the work of the loop (Section 4). On this small block `findFactor` happens to return a Hamiltonian path, but on larger sets its factor usually has several pieces, as in Fig. 14a.



black total 5 white total 5

(a) $|V| = 6$ (even), s black, t white



black total 8 white total 8

(b) $|V| = 9$ (odd), s and t both black

Figure 20: The flow network of `findFactor` for (a) an even and (b) an odd instance. Left of each panel, the source, the black points, the white points and the sink. Grey arcs from the source and to the sink have capacity 2, red arcs have capacity 1 (at s and t), and every arc between a black and a white point is an edge of V with capacity 1. Thick blue arcs carry one unit of flow. Right of each panel, the same edges on the grid. In (a) the flow is the factor `findFactor` returns, a Hamiltonian s–t path. In (b) it is another maximum flow, an L-shaped s–t path and a separate 4-cycle, which is a factor but not a Hamiltonian path. Points are black or white by the parity of $x+y$.

References

[1] Esther M. Arkin, Sándor P. Fekete, Kamrul Islam, Henk Meijer, Joseph S. B. Mitchell, Yurai Núñez-Rodríguez, Valentin Polishchuk, David Rappaport, and Henry Xiao. Not being (super)thin or solid is hard: A study of grid Hamiltonicity. *Comput. Geom.*, 42(6–7):582–605, 2009.

[2] Mikhail Basok. Dimers on Riemann surfaces and compactified free field. arXiv:2309.14522, 2023. [abstract].

[3] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *LNCS*, pages 625–635, 2021.

[4] Erik D. Demaine and Mikhail Rudoy. Hamiltonicity is hard in thin or polygonal grid graphs, but easy in thin polygonal grid graphs. arXiv:1706.10046, 2017. [abstract].

[5] Julien Dubédat. Dimers and analytic torsion I. arXiv:1110.2808, 2011. [abstract].

[6] Michael E. Fisher and John Stephenson. Statistical mechanics of dimers on a plane lattice. II. Dimer correlations and monomers. *Phys. Rev.*, 132:1411–1431, 1963.

[7] Alon Itai, Christos H. Papadimitriou, and Jayme Luiz Szwarcfiter. Hamilton paths in grid graphs. *SIAM J. Comput.*, 11(4):676–686, 1982.

[8] Jesper Lykke Jacobsen and Jané Kondev. Field theory of compact polymers on the square lattice. *Nucl. Phys. B*, 532:635–688, 1998. arXiv:cond-mat/9804048.

[9] Fatemeh Keshavarz-Kohjerdi. Hamiltonian paths in two classes of grid graphs. arXiv:1107.1780, 2011. [read].

[10] Fatemeh Keshavarz-Kohjerdi and Alireza Bagheri. Hamiltonian paths in some classes of grid graphs. *J. Appl. Math.*, 2012:Art. 475087, 2012. [abstract].

[11] Fatemeh Keshavarz-Kohjerdi and Alireza Bagheri. Hamiltonian paths in C-shaped grid graphs. arXiv:1602.07407, 2016. [read].

[12] Fatemeh Keshavarz-Kohjerdi and Alireza Bagheri. Hamiltonian paths in L-shaped grid graphs. *Theoret. Comput. Sci.*, 2016.

[13] Fatemeh Keshavarz-Kohjerdi and Alireza Bagheri. Hamiltonian (s, t)-paths in solid supergrid graphs. *Comput. Appl. Math.*, 43(3):Art. 110, 2024.

[14] M. Rouhani-Marchoobeh and Fatemeh Keshavarz-Kohjerdi. Hamiltonicity of rectangular grid graphs (meshes) with an L-shaped hole. *Rev. Un. Mat. Argentina*, 68(2):611–625, 2025. [read].

[15] Nicolau C. Saldanha, Carlos Tomei, Mario A. Casarin Jr., and Domingos Romualdo. Spaces of domino tilings. *Discrete Comput. Geom.*, 14:207–233, 1995.

[16] Asghar Asgharian Sardroud and Alireza Bagheri. An approximation algorithm for the longest cycle problem in solid grid graphs. arXiv:1502.07085, 2015. [abstract].

[17] Scott Sheffield. Computing and sampling restricted vertex degree subgraphs and Hamiltonian cycles. arXiv:math/0008231 (v1 31 Aug 2000, v2 27 Feb 2001), 2000. [read].

[18] The mathlib Community. The Lean mathematical library. In *Proc. 9th ACM SIGPLAN Int. Conf. on Certified Programs and Proofs (CPP)*, pages 367–381, 2020.

[19] William P. Thurston. Conway's tiling groups. *Amer. Math. Monthly*, 97(8):757–773, 1990.

[20] Christopher Umans and William Lenhart. Hamiltonian cycles in solid grid graphs. In *Proc. 38th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 496–505, 1997. [abstract].